

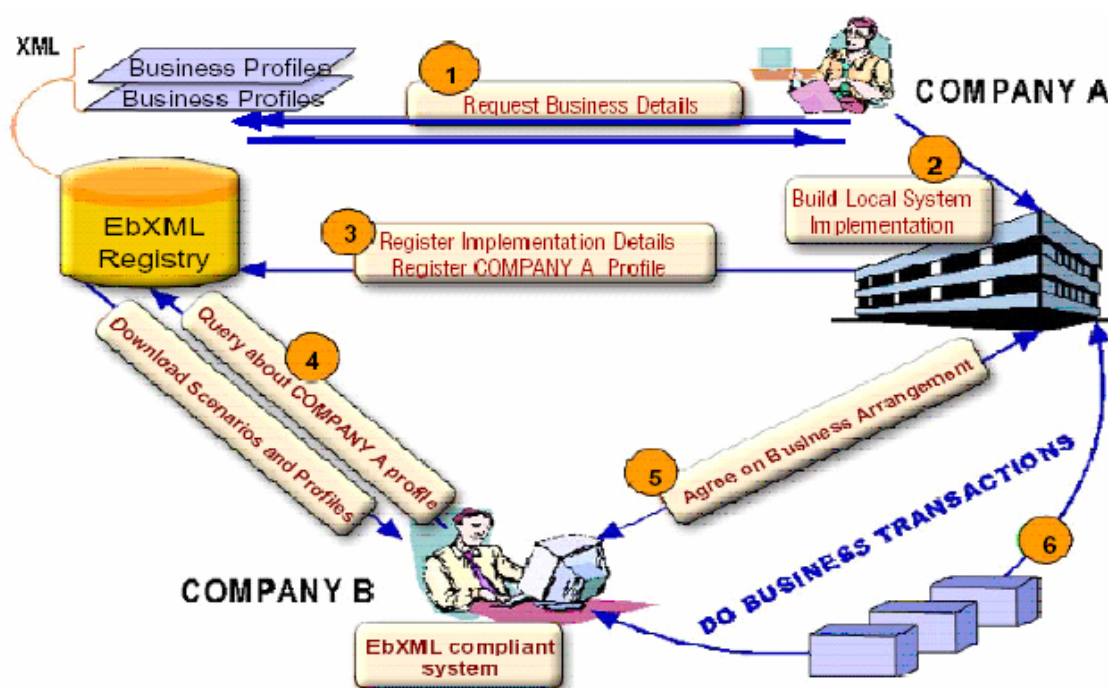
บทที่ 2

ทฤษฎีและหลักการ

2.1 เว็บเซอร์วิส (Web Service)

2.1.1 เทคโนโลยีเว็บเซอร์วิส

การพัฒนาทางเทคโนโลยีใดๆ ล้วนแล้วแต่มีการพัฒนาเป็นยุคสมัย แต่ละยุคสมัยก็จะมีการใช้เทคโนโลยีที่มีจุดสำคัญของการพัฒนาที่เด่นชัด การพัฒนาเทคโนโลยีประยุกต์ใช้งานบนเว็บก็เช่นเดียวกัน เริ่มจากยุคแรกที่เป็นแบบสแตติกเว็บเพจที่เป็นเพียงแค่ไฟล์เอชทีเอ็มแอลไว้ให้ผู้ใช้อ่านข้อมูลเฉยๆ จนสู่ยุคที่ 2 เป็นแบบไดนามิกเว็บเพจ (Dynamic Web Page) ที่สามารถปรับเปลี่ยนตามคำร้องขอของผู้ใช้งานได้ ซึ่งเป็นการเพิ่มประสิทธิภาพของเว็บเพจ จนกระทั่งในปัจจุบันมีการพัฒนาเว็บเพจเข้ามาสู่ยุคที่ 3 หรือแบบการบริการผ่านเว็บเซอร์วิส โดยจุดประสงค์เพื่อรองรับความต้องการการใช้งานที่สูงขึ้น และเป็นมาตรฐานในการทำงานร่วมกัน



รูปภาพ 2-1 ลักษณะของเว็บเซอร์วิส

เว็บเซอร์วิส คือ ซอฟต์แวร์คอมโพเนนท์ (Software Component) ที่ผู้ให้บริการนำมาสร้างเป็นแอปพลิเคชันสำหรับให้บริการการทำงานต่างๆ ให้กับผู้ขอบริการทางอินเทอร์เน็ต หรือผู้ขอบริการสามารถที่จะขอบริการจากหลายๆที่มาประกอบกันได้ โดยที่แต่ละระบบนั้นมีความเป็นอิสระจากกัน (Loosely Coupled) ซึ่งปัจจัยพื้นฐานของเว็บเซอร์วิสที่ควรมี ซึ่งมีความเกี่ยวข้องกันในทางเทคนิคและทางธุรกิจคือ

- การรวมซอฟต์แวร์ต่างระบบกัน จะต้องอนุญาตให้แต่ละระบบมีความเป็นอิสระจากกัน
- อินเทอร์เน็ตทางด้านการบริการของซอฟต์แวร์ควรจะเผยแพร่สู่สาธารณะชนและสามารถเข้าถึงได้ง่าย
- แมสเสจที่ใช้ติดต่อกันของการทำงานแบบแอปพลิเคชันกับแอปพลิเคชันต้องสอดคล้องกับมาตรฐานเปิดบนอินเทอร์เน็ต
- แอปพลิเคชันสามารถสร้างได้จากการใช้ซอฟต์แวร์คอมโพเนนท์ทั้งจากภายในและภายนอกองค์กร โดยสร้างตามการดำเนินธุรกิจหลักขององค์กร
- มีแหล่งซอฟต์แวร์คอมโพเนนท์ที่หาได้ง่าย ซึ่งช่วยเพิ่มความยืดหยุ่นในการสร้างแอปพลิเคชันที่มีคุณลักษณะเฉพาะตามกระบวนการดำเนินการทางธุรกิจ
- การนำซอฟต์แวร์จากภายนอกมาใช้ใหม่ช่วยให้เกิดการลดต้นทุนและช่วยปรับปรุงบริการให้แก่ลูกค้า

หลักการพื้นฐานของเทคโนโลยีต่างๆที่พัฒนามาสู่เว็บเซอร์วิส

- การพัฒนาโปรแกรมแบบซอฟต์แวร์คอมโพเนนท์ ตามแนวคิดของการออกแบบและพัฒนาโปรแกรมเชิงวัตถุ (Object-Oriented Concept)
- การออกแบบระบบแบบกระจายจากศูนย์กลาง (Distributed Computing) ซึ่งเป็นเป้าหมายสำคัญของการพัฒนาระบบตามสถาปัตยกรรมแบบไคลเอนต์เซิร์ฟเวอร์ (Client-Server)
- การทำอีดีไอ หรือ อิเล็กทรอนิกส์ดาต้าอินเทอร์เช인지 (Electronic Data Interchange) ซึ่งสร้างขึ้นโดยกำหนดรูปแบบและมาตรฐานของข้อมูลสำหรับการทำธุรกิจ
- การบูรณาการของซอฟต์แวร์ต่างระบบ หรือ เอนเทอร์ไพรส์ แอปพลิเคชัน อินทิเกรชัน : อีเอไอ (Enterprise Application Integration : EAI) ที่อยู่บนพื้นฐานของความต้องการใช้ข้อมูลร่วมกัน รวมทั้งการแลกเปลี่ยนข้อมูลระหว่างแอปพลิเคชันให้สามารถทำงานได้อย่างถูกต้องเหมาะสม
- รูปแบบการให้บริการซอฟต์แวร์แบบเอเอสพี (ASP) หรือ แอปพลิเคชันเซอร์วิสโพรไวเดอร์ (Application Service Provider)

2.1.2 ลักษณะของเว็บเซอร์วิส

การพัฒนาเว็บเซอร์วิสใช้สถาปัตยกรรมบริการในลักษณะที่เรียกว่า สถาปัตยกรรมของแนวคิดทางด้านบริการ หรือ เอสโอเอ (Service-Oriented Architecture : SOA) เป็นแนวคิดเบื้องต้น แอปพลิเคชัน

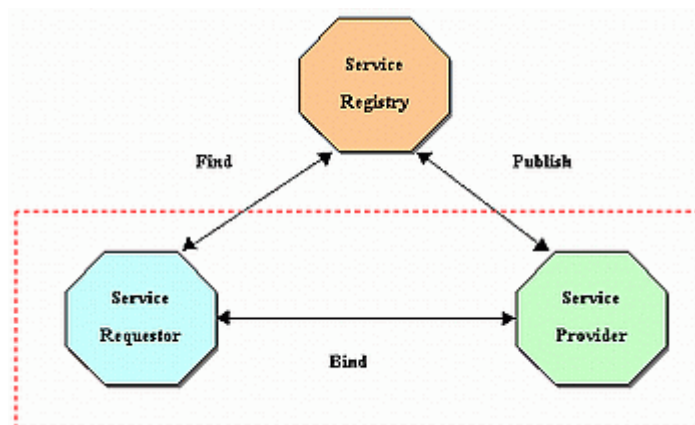
ส่วนใหญ่ในโลกของธุรกิจที่ใช้งานในปัจจุบัน เป็นแอปพลิเคชันและระบบย่อยที่ถูกสร้างขึ้น มีการทำงานที่ต้องสัมพันธ์กันอย่างไม่เป็นอิสระจากกัน การเปลี่ยนแปลงการทำงานในระบบย่อยหรือแอปพลิเคชันหนึ่ง อาจจะมีผลกระทบกับอีกแอปพลิเคชันหนึ่ง หรือบางครั้งอาจจะกระทบทั้งระบบ ทำให้การบำรุงรักษานั้นทำได้ยาก และมีต้นทุนที่สูงขึ้น รวมทั้งยังเป็นข้อจำกัดในการเชื่อมต่อกับระบบของคู่ค้าอื่นๆ

เอสโอเอมีส่วนประกอบหลัก 3 ส่วนคือ

1. ผู้ให้บริการ หรือที่เรียกว่า เซอร์วิสโพรไวเดอร์ (Service Provider)
2. ผู้ขอใช้บริการ หรือที่เรียกว่า เซอร์วิสรีเควสเตอร์ (Service Requester)
3. ตัวแทนของผู้ให้บริการ หรือที่เรียกว่า เซอร์วิสโบรกเกอร์ (Service Broker)

โดยส่วนประกอบหลักทั้ง 3 ส่วนนี้ สามารถติดต่อถึงกันได้โดยใช้ฟังก์ชันพื้นฐาน (Primary Function) คือการประกาศ (Publish) การค้นหา (Find) และการเรียกใช้ (Bind) ซึ่งฟังก์ชันทั้ง 3 มีการทำงานดังนี้

1. ผู้ให้บริการทำการประกาศบริการที่ตนเองให้บริการ ไปยังตัวแทนของผู้ให้บริการ ซึ่งตัวแทนของผู้ให้บริการจะทำการบันทึกเก็บไว้ในไดเรกทอรีของบริการ (Directory Service)
2. ผู้ขอใช้บริการจะทำการค้นหาบริการจากตัวแทนของผู้ให้บริการ
3. เมื่อพบบริการที่ต้องการแล้ว ผู้ให้บริการและผู้ขอใช้บริการจะทำการติดต่อกัน โดยผู้ขอใช้บริการจะทำการเรียกใช้บริการไปยังผู้ให้บริการนั้น



รูปภาพ 2-2 แสดงความสัมพันธ์ของผู้ให้บริการ ผู้ขอใช้บริการ และตัวแทนผู้ให้บริการ

2.1.3 เทคโนโลยีพื้นฐานของเว็บเซอร์วิส

จากแนวคิดของเอสโอเอ ถูกนำมาใช้เป็นหลักการพื้นฐานของการพัฒนาเทคโนโลยีด้านเว็บเซอร์วิส ซึ่งเทคโนโลยีที่ถูกพัฒนาขึ้นมานี้เป็นมาตรฐานเปิดบนอินเทอร์เน็ตที่เกิดจากการทำงานร่วมกันของนักวิจัย และที่ปรึกษาจากบริษัทซอฟต์แวร์ต่างๆ

- เอกซ์เอ็มแอล หรือ เอกซ์เทนซิเบิลมาร์กอัปแลงกเวจ เป็นภาษามาร์กอัปที่เป็น เท็กซ์เบส (Text-Based) ที่ใช้เป็นมาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตในปัจจุบัน ผู้ที่ทำหน้าที่รับผิดชอบ และกำหนดมาตรฐานของ เอกซ์เอ็มแอล คือ ดับบิวทีริซี หรือ เวิร์ดไวด์เว็บคอนซอร์ตียม (W3C-World Wide Web Consortium)
- โซป หรือ ซิมเปิลออเพนเจกต์แอคเซสโพรโตคอล เป็นโพรโตคอลที่สร้างบนพื้นฐานของเอกซ์เอ็มแอล และยังเป็นเมสเซจิงโพรโตคอล (Messaging Protocol) สำหรับใช้ในการแลกเปลี่ยนข้อมูลในสภาวะแวดล้อมแบบกระจายศูนย์ (Decentralized , Distributed Environment) โดยโซป ได้กำหนดเมสเซจิงโพรโตคอลระหว่างผู้ขอรับบริการกับผู้ให้บริการในการติดต่อสื่อสารกัน เช่น กำหนดให้ผู้ขอใช้บริการต้องส่งข้อมูล เช่น ข้อมูลที่ระบุฟังก์ชันและค่าพารามิเตอร์ต่างๆ ที่ต้องใช้ในแอปพลิเคชันที่ร้องขอ ส่งไปให้กับผู้ให้บริการ
- ดับบิวเอสดีแอล หรือ เว็บเซอร์วิสเดสคริปต์ซันแลงกเวจ (WSDL – Web Service Description Language) เป็นภาษาที่ใช้อธิบายคุณลักษณะการให้บริการของเว็บเซอร์วิส และวิธีการติดต่อขอรับบริการจากเว็บเซอร์วิส ดับบิวเอสดีแอลสร้างขึ้นโดยใช้ภาษาเอกซ์เอ็มแอล

2.2 เอกซ์เอ็มแอล (XML)

มาร์กอัพแลงกเวจ (Markup language) คือ ภาษาที่ใช้อธิบายความหมายของเอกสารหรือข้อมูล ในรูปของแท็ก (tag) เป็นส่วนมาก มาร์กอัพแลงกเวจมีข้อดีคือ สามารถใช้ได้กับทุกแพลตฟอร์ม

2.2.1 ลักษณะที่สำคัญของเอกซ์เอ็มแอล

เอกซ์เอ็มแอล เป็นภาษามาร์กอัพที่เป็นเท็กซ์เบสที่ใช้เป็นมาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตในปัจจุบัน โดยมีผู้ที่ทำหน้าที่รับผิดชอบและกำหนดมาตรฐานของเอกซ์เอ็มแอล คือ องค์กรดับเบิลยูทีซี

- จุดประสงค์ในการออกแบบ เพื่อกำหนดเป็นมาตรฐานในการทำเอกสารบนเครือข่ายอินเทอร์เน็ตเพื่อการแลกเปลี่ยนข้อมูลที่เป็นลำดับขั้น ดังนั้นจึงต้องมีรูปแบบที่สามารถใช้ได้ทั่วไปบนอินเทอร์เน็ต
- มีลักษณะที่ไม่ขึ้นกับแพลตฟอร์ม (Platform Independence) สามารถนำไปใช้กับคอมพิวเตอร์ระบบใด แพลตฟอร์มไหนก็ได้ เนื่องจากเอกสารเอกซ์เอ็มแอล เป็นเท็กซ์ไฟล์ (text file) ธรรมดา
- เป็นภาษาที่ได้รับการออกแบบมาเพื่อให้สามารถนิยามความหมายของข้อมูลได้ จึงมีการจัดโครงสร้างข้อมูล แบ่งข้อมูลออกเป็นหมวดหมู่และส่วนประกอบย่อย
- ไม่มีแท็กที่ถูกระบุไว้ก่อน อนุญาตให้ผู้ใช้สร้างแท็กขึ้นมาเองเพื่ออธิบายข้อมูลได้ โดยที่ผู้ใช้กำหนดแท็ก และใช้งานได้ทันที เนื่องจากแท็กเป็นอะไรก็ได้ที่ผู้ใช้กำหนดจึงทำให้เอกซ์เอ็มแอล ขยายขีดความสามารถต่อไปได้ (Extensible) และข้อมูลสามารถอธิบายความหมายข้อมูลของตัวเองได้ (Self Describe)
- ส่วนข้อมูลและส่วนการแสดงผลของเอกสารเอกซ์เอ็มแอล ถูกแยกออกจากกันอย่างชัดเจน ในเอกสารเอกซ์เอ็มแอล นั้น จะมีแต่ตัวเนื้อข้อมูล ส่วนการแสดงผลนั้น เราสามารถใช้สไตลชีต (style sheet) ได้หลายประเภท เช่น ซีเอสเอส (CSS), เอกซ์เอสแอล (XSL) เป็นต้น
- เนื่องจากเอกสารเอกซ์เอ็มแอล เป็นเพียงข้อความง่ายๆที่ประกอบด้วยแท็กบางอย่างเท่านั้น จึงสามารถสร้างเอกสารเอกซ์เอ็มแอล ด้วยเท็กซ์อีดิเตอร์ (text editor) ทั่วไปได้
- การอ่านและแปลความหมายของเอกสาร เอกซ์เอ็มแอล สามารถทำได้โดยใช้โปรแกรมที่เรียกว่า เอกซ์เอ็มแอลพาร์เซอร์ (XML Parser) ได้ ตัวอย่างของโปรแกรมประเภท เอกซ์เอ็มแอลพาร์เซอร์ เช่น เอ็มเอสเอกซ์เอ็มแอล (MSXML) ซึ่งอยู่ในอินเทอร์เน็ตเอกซ์พลอเรอร์ (Internet Explorer) ของไมโครซอฟต์ และ เจเอเอกซ์พี (JAXP) ของบริษัทซัน (Sun) เป็นต้น
- มีวิธีกำหนดโครงสร้างเอกสาร 2 วิธีคือ ดีทีดี และ เอกซ์เอ็มแอลสคริม่า (XML Schema) ซึ่งไม่ได้บังคับว่าจำเป็นต้องมีไฟล์กำหนดโครงสร้างเอกสาร แต่ถ้ามีและเอกสารเอกซ์เอ็มแอล

มีรูปแบบถูกต้องตาม ดีทีดี หรือ เอ็กซ์เอ็มแอลสคริปต์ มา จะถือว่าเอกสารเอ็กซ์เอ็มแอลนั้นมีคุณสมบัติถูกต้อง (valid)

- มีความกะทัดรัด เข้าใจง่ายและใช้ประโยชน์ได้กว้างขวาง
- สามารถใช้ได้หลายภาษาผสมกัน เนื่องจากเอ็กซ์เอ็มแอล สนับสนุน ยูนิโค้ด(UNICODE)
- ได้รับการสนับสนุนในโปรแกรมระบบฐานข้อมูลหลายๆค่าย สามารถดึงข้อมูลจากฐานข้อมูล (Database) มาอยู่ในรูปของเอ็กซ์เอ็มแอลได้

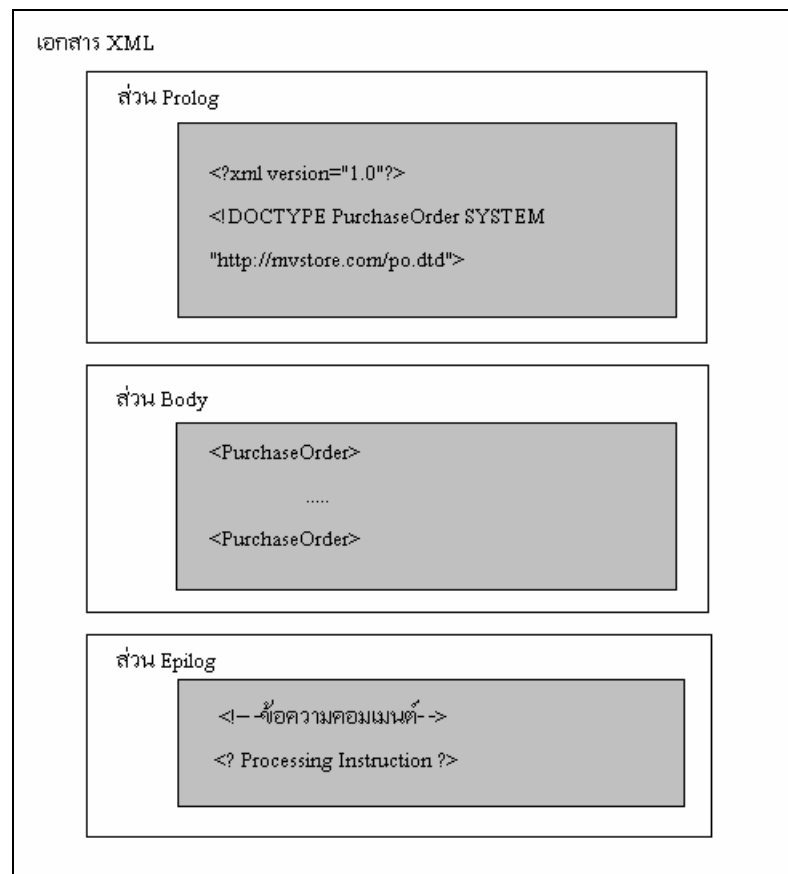
2.2.2 ประโยชน์ของเอ็กซ์เอ็มแอล

- ใช้สำหรับสร้างข้อมูลที่สามารถอธิบายความหมายของตัวเองได้ (Self-Describe Data) จากการที่เราสามารถกำหนดแท็ก มาอธิบายข้อมูลที่อยู่ภายในแท็กเองได้ ทำให้ข้อมูลมีความหมายในตัวเอง และสามารถเขียนโปรแกรมมาดึงข้อมูลไปใช้งานง่าย และคนทั่วไปก็อ่านได้เข้าใจ
- ใช้สำหรับการแลกเปลี่ยนข้อมูล (Data Exchange) ด้วยความที่เป็นไฟล์ข้อความธรรมดา ทำให้เอกสารเอ็กซ์เอ็มแอล เป็นภาษากลางที่สามารถใช้ได้ทุกแพลตฟอร์ม จึงไม่แปลกที่เราจะสามารถแลกเปลี่ยนข้อมูลข้ามแพลตฟอร์มได้
- เป็นรูปแบบข้อความในการสื่อสาร (Messaging Format) ระหว่างแอปพลิเคชันหรือโปรแกรม เอ็กซ์เอ็มแอลเป็นรูปแบบการสื่อสารระหว่างองค์ประกอบต่างๆ ตามแนวคิดของเว็บเซอร์วิส นอกจากเอ็กซ์เอ็มแอลแล้ว ยังมีมาตรฐานต่างๆที่เป็นส่วนสำคัญในการทำงานของเว็บเซอร์วิส เช่น โซบ หรือ ซิมเปิ้ลออปเจกต์เอกเซสโปรโตคอล และยูดีดีไอ หรือ ยูนิเวอร์ซัลเดสคริปต์ซันคิสคัพเวอร์แอนด์อินทิเกรชั่น ซึ่งทั้ง โซบ และ ยูดีดีไอ ล้วนมีพื้นฐานมาจาก เอ็กซ์เอ็มแอล เช่นกัน
- ประโยชน์ในเชิงเทคโนโลยีอินเทอร์เน็ตและการพัฒนาเว็บ
- เป็นรากฐานของภาษาใหม่ๆในการพัฒนาเว็บ ภาษาใหม่ๆนี้ได้แก่ ภาษา เอ็กซ์เอ็มแอล (XHTML), แมทเอ็มแอล (MathML คือ กลุ่มของแท็กเพื่อใช้นิยามเครื่องหมายในทางคณิตศาสตร์ขั้นสูง), วีเอ็มแอล (VML คือภาษาที่ใช้วาดรูปกราฟิกเพื่อแสดงผ่านเว็บเบราว์เซอร์) และ ดับเบิลเอ็มแอล (WML คือ ภาษาที่ใช้ในการสร้าง แวพไซด์ (Wap Site)) เป็นต้น
- ใช้ในแวดวงธุรกิจแบบบีทูบี (B2B = Business to Business) กรณีนี้จะต้องใช้ภาษาเฉพาะ อย่างเช่น ซีเอ็มเอ็กซ์แอล (cMXL = Commerce XML), เอ็กซ์ซีบีแอล (xCBL = XML Common Business Language) เป็นต้น โดยมีแท็กที่ใช้สนับสนุนการจัดการเกี่ยวกับแคตตาล็อกสินค้า และธุรกรรมเกี่ยวกับอีคอมเมิร์ซ

2.2.3 โครงสร้างของภาษาเอ็กซ์เอ็มแอล

โครงสร้างของเอกสารเอ็กซ์เอ็มแอล ประกอบด้วย 3 ส่วน

- โปรล็อก (Prolog) แบ่งเป็น ดีคลิเรชัน (Declaration) และ ดีคไทป์เดคลิเรชัน (Document Type Declaration)
- บอดี้ (Body) เป็นส่วนของเนื้อเอกสารจริงๆ ได้แก่ข้อความหรือข้อมูลในเอกสาร และ แท็กที่นิยามข้อความหรือข้อมูลเหล่านั้น
- อีพิล็อก (Epilog) คือส่วนที่เป็นข้อความจำพวกคอมเมนต์ (Comment) และ พีไอ (PI) หรือ โปรเซสซิงอินสตรักชัน (Processing Instruction) จริงๆแล้วส่วนนี้ไม่จำเป็นต้องอยู่บริเวณล่างสุดของเอกสารดังในภาพ แต่คอมเมนต์ และ พีไอ จะแทรกอยู่ในส่วนบอดี้ของเอกสาร แต่ในภาพนี้แยกไว้ด้านล่างเพื่อให้เห็นว่าในเอกสาร เอ็กซ์เอ็มแอล หนึ่งๆ มีองค์ประกอบ 3 ส่วน



รูปภาพ 2-3 ส่วนประกอบของเอกสารเอ็กซ์เอ็มแอล

2.2.4 การแปลความหมายของเอ็กซ์เอ็มแอลด้วยเอ็กซ์เอ็มแอลพาร์เซอร์

ในการพัฒนาแอปพลิเคชันโดยมีการนำเอ็กซ์เอ็มแอล ไปใช้งานนั้น สิ่งหนึ่งที่ผู้พัฒนาจำเป็นต้องทำการพาส (Parse) เอกสารเอ็กซ์เอ็มแอล เนื่องจากโปรแกรมมองเห็นเอกสารเอ็กซ์เอ็มแอล เป็นเพียงอักขระธรรมดาเท่านั้น เราจึงต้องมีการเขียนโปรแกรมเพื่อให้ แอปพลิเคชัน สามารถเข้าใจได้ว่าเอกสารเอ็กซ์เอ็มแอล ของเรามีการจัดโครงสร้างเป็นอย่างไร โดยเราเรียกโปรแกรมนั้นว่าพาร์เซอร์

การจำแนกชนิดของเอ็กซ์เอ็มแอลพาร์เซอร์ แบ่งได้เป็นหลายวิธี แต่หลักเกณฑ์ที่นิยมกันมี 2 วิธี คือ

1. เอ็กซ์เอ็มแอลพาร์เซอร์ แบบValidating และ แบบNon-Validating ซึ่งหมายถึง พาร์เซอร์ที่มีการตรวจสอบคุณสมบัติเวลฟอร์มเนส และคุณสมบัติValidของเอกสาร ส่วนเอ็กซ์เอ็มแอลพาร์เซอร์ แบบNon-Validating หมายถึง เอ็กซ์เอ็มแอลพาร์เซอร์ที่ไม่มีการตรวจสอบคุณสมบัติValid
2. เอ็กซ์เอ็มแอลพาร์เซอร์ แบบที่รองรับ คือคิควเมนต์ออบเจกต์โมเดลลิง (DOM – Document Object Model) และแบบที่รองรับที่เรียกว่า แซก หรือ ซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอล (SAX – Simple API for XML)

การแปลเอกสารโดยซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอล

ซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอลไม่ได้เป็นมาตรฐานของหน่วยงานใด แต่เป็นวิธีที่นักพัฒนากลุ่มหนึ่งคิดค้นขึ้น และนำไปใช้กันมากจนได้รับการยอมรับทั่วไป ปัจจุบันเอ็กซ์เอ็มแอลพาร์เซอร์ จำนวนมากล้วนแต่รองรับซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอล กันแล้ว

ซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอลจะไม่โหลดเอกสารเอ็กซ์เอ็มแอล เข้ามาในหน่วยความจำจึงไม่มีการสร้างภาพรวมของเอกสารไว้ก่อนว่าเอกสารนั้นมีโครงสร้างเป็นอย่างไร วิธีการของมันคือเมื่อแอปพลิเคชันมีการร้องขอมา มันก็จะอ่านเอกสารเอ็กซ์เอ็มแอลไล่ตั้งแต่ต้นเอกสารและตัวพาร์เซอร์จะสร้างอีเวนต์ (Event) ให้กับจุดต่างๆที่สำคัญของเอกสารทุกๆจุด เช่นทุกๆ เอลเลเมนต์เปิด เอลเลเมนต์ปิด ทุกๆแอททริบิว เป็นต้น ลักษณะการทำงานเช่นนี้เรียกว่า อีเวนต์ไดรเวอร์พาร์เซอร์ (Event-Driven Parser)

ในทางปฏิบัติแล้วซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอล มักไม่ได้มีการแปลเอกสารทั้งเอกสาร เมื่อเราได้ข้อมูลที่ต้องการแล้วเราก็จะหยุด วิธีการแปลแบบซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอล จึงทำงานได้รวดเร็ว และเขียนโปรแกรมได้ง่าย

ข้อดีของซิมเปิ้ลเอพีไอฟอร์เอ็กซ์เอ็มแอล

- ไม่เปลืองหน่วยความจำ เพราะไม่ได้โหลดทั้งเอกสารมาเก็บไว้เหมือน ค็อกคิวเมนต์ออบเจกต์โมเดลลิง จึงเหมาะกับกรณีที่เอกสารเอ็กซ์เอ็มแอล มีขนาดใหญ่ และเครื่องคอมพิวเตอร์มีหน่วยความจำไม่มาก

- ซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอลเข้าถึงข้อมูลได้ง่ายกว่า คือคคิวเมนต์ออบเจกต์โมเดลลิง เพราะการทำงานของ แชก จะมองเพียงจุดเริ่มต้นและจุดสิ้นสุดของเอกสารและของ เอลเลเมนต์ และจะสนใจเฉพาะสิ่งที่ต้องการเท่านั้น จึงเหมาะกับกรณีต้องการใช้ข้อมูลจากเอลเลเมนต์เพียงไม่กี่ตัวในเอกสาร ซึ่งถ้าใช้วิธี คือคคิวเมนต์ออบเจกต์โมเดลลิง จะเป็นงานที่กินแรงเกินความจำเป็น
- สามารถเขียนโปรแกรมได้ง่ายกว่า เพราะมองข้อมูลในลักษณะเส้นตรง

ข้อเสียของซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล

- ไม่สามารถบอกความสัมพันธ์ของข้อมูลได้
- ไม่สามารถใช้ข้อมูลแบบสุ่มได้ เพราะไม่มีข้อมูลใดเหลืออยู่ในหน่วยความจำหลังจากที่ซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล ได้ทำการประมวลผลและสร้างอ็อบเจกต์ที่มีความสัมพันธ์กันเสร็จแล้ว
- ยากที่จะสร้างฟังก์ชัน ในการค้นหาข้อมูล
- อ่านได้เพียงอย่างเดียว ไม่สามารถเขียนหรืออัปเดตข้อมูลได้
- เป็นวิธีที่ไม่มีประสิทธิภาพถ้าต้องการเข้าถึงข้อมูลเดิมบ่อยครั้ง

2.3 ซิมเบิลออบเจกต์แอคเซสโปรโตคอล (Simple Object Access Protocol)

เนื่องจากจุดประสงค์หลักของการใช้งานเว็บเซอร์วิส เราต้องการให้แอปพลิเคชัน มีการทำงานกับแอปพลิเคชันที่ทำงานอยู่บนเครื่องอื่นโดยผ่านทางเครือข่าย ซึ่งเทคโนโลยีที่มีอยู่ปัจจุบันที่ใช้การสื่อสารระหว่างออบเจกต์ในระยะไกล หรือ อาร์พีซี (Remote Procedure Calls : RPC) เช่น ดิคอม (DCOM) , อีเจบี (EJB) หรือคอบร้า (COBRA) นั้น ไม่ได้ถูกออกแบบมาใช้สำหรับโปรโตคอลเอชทีทีพี

เทคนิคอาร์พีซีของเทคโนโลยีที่กล่าวข้างต้นต่างก็มีปัญหาในด้านการนำมาใช้งานในแง่ของความเข้ากันได้ของการเรียกใช้งานข้ามเทคโนโลยี เนื่องจากเป็นเทคโนโลยีเฉพาะของแต่ละค่าย (ยกเว้น คอบร้า) ผู้พัฒนาระบบจะต้องพัฒนาโปรแกรมที่มีความซับซ้อน และยังมีปัญหาในส่วนของไฟร์วอลล์ และพร็อกซีเซิร์ฟเวอร์ ด้วยเนื่องจากโดยปกติเซิร์ฟเวอร์จะปิดการสื่อสารที่ไม่ใช่โปรโตคอลเอชทีทีพีออกไปเพื่อความปลอดภัยของระบบที่มีการติดต่อสื่อสารกับภายนอก

ดังนั้นทางเลือกของการสื่อสารที่จะนำมาใช้ในการทำบริการเว็บเซอร์วิส คือ ให้ทำงานอยู่บนโปรโตคอลเอชทีทีพี ซึ่งโซบ นอกจากจะทำงานบนโปรโตคอลเอชทีทีพีแล้ว ยังเป็นมาตรฐานเปิดที่จะทำให้สามารถติดต่อสื่อสารกับคอมพิวเตอร์ที่มีความแตกต่างทางระบบปฏิบัติการ, เทคโนโลยี รวมไปถึงภาษาที่ใช้ในการพัฒนา

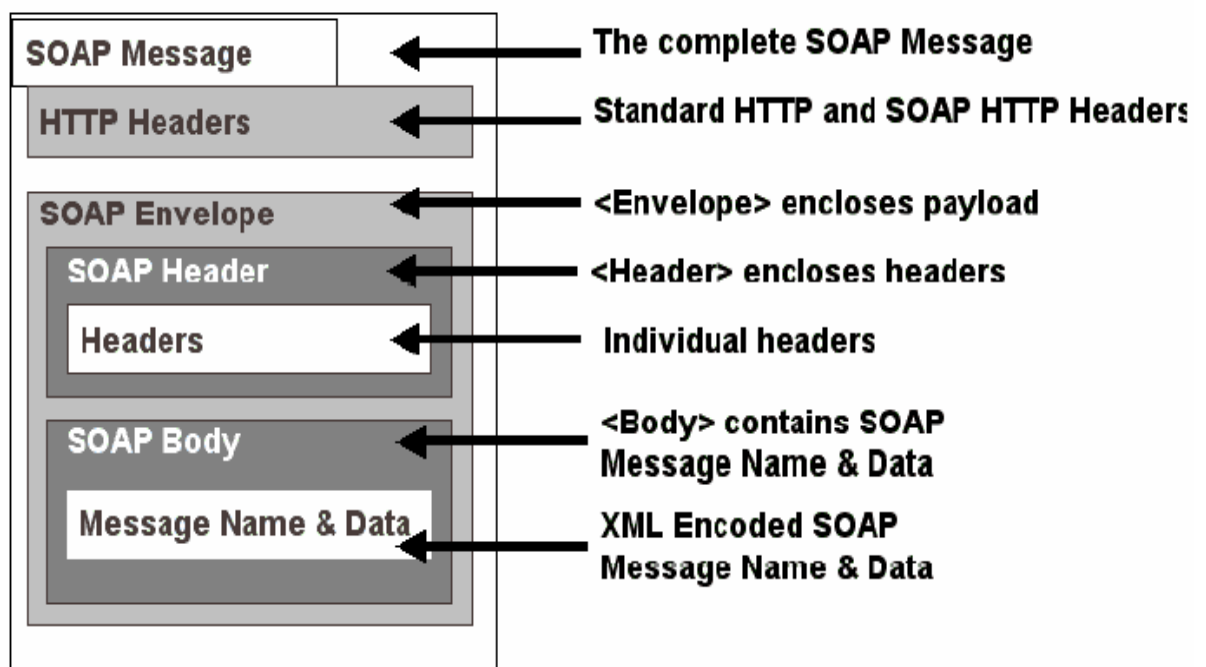
โซบ หรือ ซิมเบิลออบเจกต์แอคเซสโปรโตคอล (SOAP = Simple Object Access Protocol) เป็นโปรโตคอลที่ใช้ เอ็กซ์เอ็มแอล เป็นพื้นฐาน เพื่อให้ซอฟต์แวร์คอมพิวเตอร์ และแอปพลิเคชันสามารถติดต่อกันผ่านเอชทีทีพี ซึ่งเป็นมาตรฐานอินเทอร์เน็ตโปรโตคอลได้ ซึ่งข้อดีของโซบก็คือโซบไม่ขึ้นกับคอมพิวเตอร์เทคโนโลยี และภาษาการเขียนโปรแกรมใดๆ สามารถเขียนได้ง่ายและขยายเพิ่มเติมได้

2.3.1 ส่วนประกอบของโซบ

โซบแมสเชจ ใช้ไวยากรณ์ของเอ็กซ์เอ็มแอล ในการสร้างประกอบด้วย 3 เอลเลเมนต์มาตรฐาน คือ โซบเอนวิล็อพ, โซบเฮดเดอร์, โซบบอดี

โซบนั้นประกอบด้วยกัน 3 ส่วนคือ

1. โซบเอนวิล็อพ (SOAP Envelop) จะเป็นส่วนสำหรับใช้ในการระบุสิ่งที่อยู่ในเอกสาร ว่า จะต้องจัดการอย่างไร และบอกถึงความจำเป็นในการใช้งาน
2. โซบเอนโค้ดดิ้งรูล (SOAP Encoding Rule) จะเป็นส่วนสำหรับกำหนดกลไกที่ใช้ในการ แลกเปลี่ยนข้อมูล ว่ามีข้อตกลงยังงในการแลกเปลี่ยนข้อมูล
3. โซบอาร์พีซีรีพรีเซนเทชัน (SOAP RPC Representation) จะเป็นส่วนสำหรับนิยามรูปแบบ รีโมท โพรซีเยอร์คอล (Remote Procedure Call) และ การตอบสนอง (Response)



รูปภาพ 2-4 โครงสร้างของโซบ

โซบแมสเชจ จะต้องเป็นไปตามกฎนี้

- เอนวิล็อพเป็นเอลเลเมนต์ที่อยู่บนสุด ต้องมีเอลเลเมนต์นี้เสมอ
- เฮดเดอร์อาจจะมีหรือไม่มีก็ได้ แต่ถ้ามีต้องเป็นชายเอลเลเมนต์ (Child Element) แรกของเอนวิล็อพ
- บอดีต้องเป็นชายเอลเลเมนต์แรกของเอนวิล็อพ หรือ เฮดเดอร์

ตัวอย่างของ โซบเมสเชจ ตัวอย่างนี้คือ GetLastTradePrice โซบรีเควส (SOAP Request) ที่ส่งไปยัง StockQuote เซอร์วิสประกอบด้วยสตริงพารามิเตอร์ ซิมโบ (Symbol) และ คีนค่าโฟลต (Float) กลับมากับ โซบเรสสปอน (SOAP Response) โซบเอนวิลอพอเอลเลอเมนที่อยู่ในจุดบนสุดของเอกสารเเอ็กซ์เอ็มแอล เเอ็กซ์เอ็มแอลเนมสเปซใช้เพื่อป้องกันการสับสนของโซบไอดี้นทิฟาย (SOAP Identifier)โซบรีเควสจะเป็นดังนี้

POST /StockQuote HTTP/1.1

HOST:www. Stockquoteserver.com

Content-Type: text/xml; charset="utf-8"

Content-Length: nnnn

SOAPAction: "Some-URI"

<SOAP-ENV:Envelope

xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"

SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">

<SOAP-ENV:Body>

<m:GetLastTradePrice xmlns:m="Some-URI">

<symbol>DIS</symbol>

</m:GetLastTradePrice>

</SOAP-ENV:Body>

</SOAP-ENV:Envelope>

รูปภาพ 2-5 ตัวอย่างของโซบรีเควส

โซบเรสสปอนจะเป็นดังนี้

HTTP/1.1 200 OK

Content-Type: text/xml; charset="utf-8"

Content-Length: nnnn

<SOAP-ENV:Envelope

xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"

```

SOAP-encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<m:GetLastTradePriceREsponse xmlns:m="Some-URI">
<Price>34.5</Price>
</m:GetLastTradePriceREsponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

รูปภาพ 2-6 ตัวอย่างของโซบเรสสพอน

นอกจากนั้นในโซบเมสเชจจะมีการใช้ เอ็กซ์เอ็มแอลเนมสเปซ ทุกๆ เอลเลอเมนต์ในเอกสารจะขึ้นต้นด้วยเนมสเปซซึ่งเนมสเปซจะถูกกำหนดโดยใช้เอ็กซ์เอ็มแอลเอนเอสแอททริบิว (xmlns attribute) ดังนี้

SOAP-Envelope

xmlns:SOAP-ENV="<http://schemas.xmlsoap.org/soap/envelope/>"

รูปภาพ 2-7 เอ็กซ์เอ็มแอลเอนเอสแอททริบิว

แต่หากไม่ต้องการให้ขึ้นต้นด้วยเนมสเปซทำได้ดังนี้

<Envelope

xmlns="<http://schemas.xmlsoap.org/soap/envelope/>">

รูปภาพ 2-8 เอ็กซ์เอ็มแอลเอนเอสแอททริบิวที่ไม่ขึ้นต้นด้วยเนมสเปซ

2.3.1.1 เอนวิล๊อฟ (Envelope)

เอนวิล๊อฟเอลเลอเมนต์เป็นเอลเลอเมนต์บนสุดของเอกสารเอ็กซ์เอ็มแอลที่ใช้แสดงเมสเชจ อาจประกอบด้วยแอททริบิว คือ เอนวิล๊อฟเนมสเปซ และเอนโค้ดดิ้งสไตล์ (EncodingStyle)

- เอนวิล๊อฟเนมสเปซ (Envelope Namespace) โซบเมสเชจจะแสดงเวอร์ชัน โดยใช้เนมสเปซของเอนวิล๊อฟเอลเลอเมนต์เนมสเปซ (Element Namespace) จะถูกนำไปใช้ขึ้นต้น เอนวิล๊อฟ ,เฮดเดอร์ และบอดีเอลเลอเมนต์ (Body Element)
- เอนโค้ดดิ้งสไตล์แอททริบิว (EncodingStyle Attribute) ใช้แสดงวิธีซีรียัลไรเซชัน (Serialization) ของโซบเมสเชจ แอททริบิวนี้สามารถมีได้ในทุกเอลเลอเมนต์ และจะมีผลกับคอนเทน (Content) ของเอลเลอเมนต์นั้นและชาดส์เอลเลอเมนต์ทั้งหมดของที่ได้ประกาศแอททริบิว

2.3.1.2 บอดี (Body)

เป็นส่วนของข้อมูลที่ใช้ในการแลกเปลี่ยน โดยทั่วไปข้อมูลจะเป็นการมาร์แชลลาร์พีซีคอล (Marshall RPC Call) และเออเรอร์รีพอร์ตชาลด์เอลเลอเมนต์ (Error Report Child Element) ของบอดีเอลเลอเมนต์จะถูกเรียกว่า บอดีเอนทรี (Body Entry)

บอดีเอนทรีจะต้องเป็นไปตามกฎนี้

- บอดีเอนทรีจะต้องแสดงด้วยชื่อเต็มประกอบด้วยเนมสเปซยูอาร์ไอ (Namespace URI) และ ชื่อของมัน (Local Name)
- โซบเอนโค้ดดิ้งสไตล์แอททริบิวต์ (SOAP EncodingStyle Attribute) อาจจะมีได้ เพื่อแสดงถึงวิธี เอนโค้ด (Encode) ของบอดีเอนทรีนั้น

2.3.1.3 เฮดเดอร์ (Header)

เป็นส่วนเพิ่มเติมของเมสเสจสามารถประกอบด้วยอินฟอร์เมชันที่ระบุไปยังแอปพลิเคชัน ส่วนเพิ่มเติมนี้อาจนำไปใช้ยืนยันตัวตนเป็นออปชัน (Authentication), ทรานแซกชันแมนเนจเมนต์ (Transaction Management) เป็นต้น ทุก ๆ ซาเอลเลอเมนต์ของเฮดเดอร์เอลเลอเมนต์ จะถูกเรียกว่า เฮดเดอร์เอนทรี (Header Entry)

เฮดเดอร์เอนทรีจะต้องเป็นไปตามกฎดังนี้

- เฮดเดอร์เอนทรีจะต้องแสดงด้วยชื่อเต็มประกอบด้วยเนมสเปซยูอาร์ไอ และ ชื่อของมัน
- อาจมีโซบเอนโค้ดดิ้งสไตล์แอททริบิวต์ที่ใช้สำหรับเฮดเดอร์เอนทรี
- โซบมัสต์อันเดอร์สแตนดแอททริบิวต์ (SOAP mustUnderstand Attribute) และ โซบแอคเตอร์
- แอททริบิวต์ (SOAP actor attribute) จะมีหรือไม่ก็ได้ เพื่อใช้บอกว่าจะจัดการกับเอนทรี นั้น อย่างไรและโดยใคร

```
<soap:Header>
  <m:local xmlns:m="http://www.w3schools.com/local"
    soap:actor="http://www.w3schools.com/appml/">
    <m:language>en</m:language>
    <m:currency>USD</m:currency>
  </m:local>
</soap:Header>
```

รูปภาพ 2-9 ตัวอย่างของโซบเฮดเดอร์

2.3.1.3.1 โซบแอดคเตอร์แอททริบิว

โซบแมสเชจสามารถถูกส่งไปยังปลายทางผ่านตัวกลางของโซบ (SOAP Intermediary) ตัวกลางของโซบ คือ แอปพลิเคชันที่มีความสามารถทั้งรับและส่งต่อโซบแมสเชจ ในการส่งต่อตัวกลางจะต้องไม่ส่งต่อเฮดเดอร์เอดเลอเมนต์ไปให้กับแอปพลิเคชัน โดยตัวกลางนี้จะถูกกำหนดเป็นยูอาร์ไอ ถ้าไม่มีแอททริบิวนี้ หมายถึง ผู้รับโซบแมสเชจเป็นปลายทางสุดท้าย

2.3.1.3.2 โซบอันเดอร์สแตนแอททริบิว

ใช้แสดงว่า เฮดเดอร์เอนทรีนี้จำเป็นหรือเป็นเพียงออปชันสำหรับผู้รับที่จะนำไปประมวลผล ค่าของมันคือ “1” หรือ “0” ถ้าไม่มีการระบุจะมีค่าเท่ากับ “0” โดย “0” หมายถึงเป็นออปชันและ “1” หมายถึงจำเป็น โดยจะต้องประมวลผลให้ถูกต้องตามซีเมนติก (Semantic) หรือ ต้องไม่เฟว (Fail) เช่น แมสเชจเป็นส่วนหนึ่งของทรานแซคชัน ถ้าปลายทางไม่รองรับทรานแซคชัน จะต้องไม่ประมวลผลและส่งผลผิดพลาดกลับไปในกรณีที่มีสอันเดอร์สแตนแอททริบิว เป็น “1” แต่ถ้าเป็น “0” จะยังคงประมวลผลต่อไป

2.3.2 โซบฟลอเอลเลอเมนต์ (SOAP Fault Element)

ข้อความแสดงความผิดพลาดจากแอปพลิเคชันของโซบจะเก็บอยู่ในฟลอเอลเลอเมนต์ ซึ่งถ้ามีจะต้องปรากฏในบอดีเอดเลอเมนต์เพียงครั้งเดียวในโซบแมสเชจ ตัวอย่างอาจเป็นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:MustUnderstand</faultcode>
      <faultstring>Mandatory Header error.</faultstring>
      <faultactor>http://www.wrox.com/heroes/endpoint.asp</faultactor>
      <detail>
        <w:source xmlns:w="http://www.wrox.com/">
          <module>endpoint.asp</module>
          <line>203</line>
        </w:source>
      </detail>
    </soap:Fault>
  </soap:Body>
```

รูปภาพ 2-10 ตัวอย่างของโซบแมสเชจที่มีความผิดพลาด

โชนฟอลเอลเลอเมนนที่มีเอลเลอเมนนที่น้อย ๆ ดังตารางนี้

Sub Element	Description
<faultcode>	โค้ดที่ระบุถึงการ error
<faultstring>	ข้อความการ error
<faultactor>	ใครเป็นสาเหตุของการ error
<detail>	ระบุอินฟอร์เมชันของการ error

ตาราง 2-1 ตัวอย่างของเอลเลอเมนนที่น้อยๆ

ค่าของฟอลโค้ด (Fault Code) สามารถมีค่าได้ดังตารางนี้

Error	Description
VersionMismatch	namespace ภายใน SOAP Envelope element ไม่ถูกต้อง
MustUnderstand	child element ของ Header element กับ mustUnderstand attribute ที่มีค่า "1" ผู้รับไม่รับรอง
Client	แมสเซจที่รูปแบบไม่ถูกต้อง หรือมีอินฟอร์เมชันไม่ถูกต้อง
Server	เกิดปัญหากับเซิร์ฟเวอร์ ไม่สามารถประมวลผลได้

ตาราง 2-2 ตัวอย่างของค่าฟอลโค้ด

2.3.3 โซบเอดโค้ดดิ้ง (SOAP Encoding)

โซบเอนโค้ดดิ้ง มีวิธีการแมพ (Map) จากไทพ์ของโปรแกรมมิ่งไปเป็นเอ็กซ์เอ็มแอล 2 วิธี คือ จากภายนอก โดยใช้ดัตต้าบิวเอสดีแอล ที่บอกถึงไทพ์ของข้อมูลที่ได้รับหรือส่ง หรือใช้เอ็กซ์เอสไอไทพ์แอททริบิว (xsi:type Attribute) ในกรณีทีภาษาที่ใช้ไม่รองรับ ดัตต้าบิวเอสดีแอล โดยทั้ง 2 วิธีจะใช้เอ็กซ์เอ็มแอลสคริปต์มาในการระบุไทพ์ ตัวอย่างของการใช้เอ็กซ์เอสไอจะเป็นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>
    <m:MixedMessage xmlns:m="http://www.wrox.com/mix/">
      <param1 xsi:type="xsd:string">OU812</param1>
      <param2 xsi:type="xsd:integer">2001</param2>
      <param3 xsi:type="xsd:double">3.14159</param3>
    </m:MixedMessage>
  </soap:Body>
</soap:Envelope>
```

รูปภาพ 2-11 ตัวอย่างของการใช้เอ็กซ์เอสไอ

2.3.4 โขบในเอชทีทีพี

ในการส่งโขบผ่านทางเอชทีทีพีจะต้องใช้คอนเทนไทป์ (Content-Type) เป็นเท็กเอ็ชเอ็มแอล (Text/xml) แต่ในโขบรีเควสจะต้องมี เฮดเคอร์โขบแอกชั่น (Header SOAPAction) ภายในเอชทีทีพีเฮดเคอร์ (HTTP Header) ตัวโขบแอกชั่น (SOAPAction) จะเป็นตัวบอกให้เซิร์ฟเวอร์รู้ว่าเอชทีทีพีโอส (HTTP Post) นั้นเป็นโขบแมสเชจ และค่าของเฮดเคอร์ คือ ยูอาร์ไอที่แสดงถึงจุดหมายของโขบแมสเชจส์ วนโขบเรสสปอนจะต้องมีสเตตัสโค้ด (Status Code) ตามมาตรฐานของเอชทีทีพี โดย 200 - 299 แสดงว่าสำเร็จ แต่ถ้าเรสสปอนแมสเชจเป็นการฟอล (Fault) แล้วสเตตัสโค้ดจะต้องเป็น 500 ซึ่งแสดงถึง อินเทอร์ นัลเซิร์ฟเวอร์เออเรอ (Internal Server Error) ตัวอย่าง ของเรสสปอนที่มีสเตตัสโค้ดเป็น 500 อาจเป็นดังนี้

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml
Content-Length: ###
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStytle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:VersionMismatch</faultcode>
      <faultstring>The SOAP namespace is incorrect.</faultstring>
      <faultactor>http://www.wrox.com/endpoint.asp</faultactor>
      <detail>
        <w:errorinfo xmlns:w=" http://www.wrox.com/">
          <desc>The SOAP namespace was blank.</desc>
        </w:errorinfo>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

รูปภาพ 2-12 ตัวอย่างของเรสสปอนที่มีสเตตัสโค้ดเป็น 500

2.3.5 โขบทูลคิต

โขบทูลคิต คือ เครื่องมือที่จะทำหน้าที่ในการประมวลผลโซบริเวส และส่งเรสสปอนกลับไปที่ไคลเอนต์โดยโขบทูลคิต แต่ละตัวใช้ได้บนแพลตฟอร์มที่ต่างกัน รองรับเทคโนโลยีเว็บเซอร์วิสที่ต่างกัน บางตัวอาจจะรองรับดับบีวเอสดีแอล หรือ ยูคียไอ ในขณะที่บางตัวรองรับอย่างใดอย่างหนึ่งหรือไม่รองรับทั้งสองอย่าง และถึงแม้ว่าโขบจะเป็นอิสระต่อแพลตฟอร์ม แต่ในโขบทูลคิตบางตัวยังไม่สามารถทำงานข้ามผลิตภัณฑ์ (Interoperability) ได้ เนื่องจากรองรับเทคโนโลยีของเว็บเซอร์วิสไม่เหมือนกัน เช่น เอ็กซ์เอสไอในบางผลิตภัณฑ์จะบังคับให้โขบแมสเชจจะต้องระบุไทป์ด้วยเอ็กซ์เอสไอ หากไม่มีก็จะเป็นฟอล (Fault) ซึ่งในปัจจุบันแต่ละผลิตภัณฑ์ก็พยายามพัฒนาให้สามารถทำงานด้วยกันได้

ชื่อ	แพลตฟอร์ม	แหล่งที่มา
4s4c	COM	http://www.4s4c.com
A SOAP for RPC NT Service	COM	http://www.whitemesa.com
Apache Axis	Java	http://xml.apache.org/axis
Apache SOAP	Java	http://xml.apache.org/soap
CapeConnect	Java	http://www.capeclear.com
Glue	Java	http://www.themindelectric.com
IBM Web Service Toolkit	Java	http://www.alphaworks.ibm.com
IdooXoop for Java and C++	Java, C++	http://www.zvon.org
Microsoft SOAP Toolkit	COM	http://www.microsoft.com
PacketSOAP	COM	http://www.packetsoap.com
SOAP::Lite	Perl	http://www.soaplite.com
Visual Studio .NET	COM	http://www.microsoft.com

ตาราง 2-3 ตัวอย่างของโขบทูลคิต

2.3.6 โครงสร้างภายในของโขบทูลคิตของไมโครซอฟต์

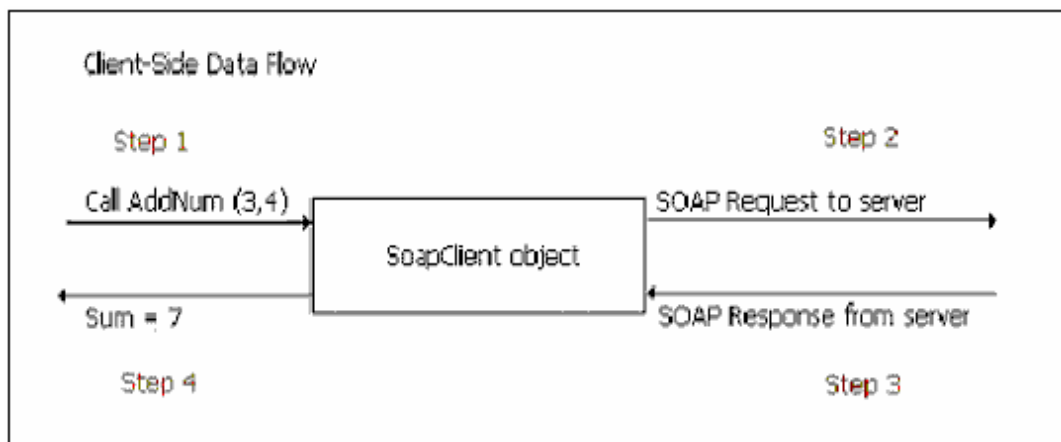
เว็บเซอร์วิสเดสคริปต์ชั้นแลงแกจ หรือ ดับบีวเอสดีแอลเป็นรูปแบบหนึ่งของเอ็กซ์เอ็มแอลเพื่อใช้อธิบายว่าเซิร์ฟเวอร์นั้นมีเซอร์วิส และออปเปอร์เรชั่น (Operation) ภายในเซอร์วิสอะไรให้ใช้บ้าง ซึ่งออปเปอร์เรชั่นภายในเซอร์วิสนั้นจะอธิบายรูปแบบที่ไคลเอนต์ต้องใช้ในการเรียกออปเปอร์เรชั่น

เว็บเซอร์วิสเมต้าแลงแกจ หรือ ดับบีวเอสเอ็มแอลไฟล์ (WSML File) เป็นส่วนที่เพิ่มขึ้นมาจากไฟล์ดับบีวเอสดีแอลเพื่อใช้ในการเชื่อมโยงแต่ละออปเปอร์เรชั่นในเซอร์วิสกับคอมมอบเจกต์ (COM Object) โดยดับบีวเอสเอ็มแอลไฟล์จะพิจารณาว่าคอมมอบเจกต์ตัวใดที่จะต้องถูกโหลดขึ้นมาเมื่อมีการเรียกใช้ออปเปอร์เรชั่นในเซอร์วิส

ทั้งไฟล์ดับเบิลยูเอสบีแอล และดับเบิลยูเอสบีเอ็มแอลจะต้องอยู่ที่เซิร์ฟเวอร์ โดยไคลเอนต์ที่ต้องการที่จะส่งโซบรีเคาส์ไปที่เซิร์ฟเวอร์จะต้องได้รับดับเบิลยูเอสบีแอลจากเซิร์ฟเวอร์ก่อน เพื่อที่จะรู้รูปแบบของโซบรีเคาส์ในการติดต่อ

การสร้างโซบรีเคาส์ไคลเอนต์ออบเจกต์ (SOAP Client Object)

ทางฝั่งไคลเอนต์เมื่อแอปพลิเคชันส่งแมสเสจไปที่โซบรีเคาส์ไคลเอนต์ออบเจกต์เพื่อเรียกใช้ ออปเปอเรชันหนึ่งๆนั้น (ตามรูปนี้ คือการบวกเลข 2 ตัว) โซบรีเคาส์ไคลเอนต์ออบเจกต์ จะทำการสร้างโซบรีเคาส์ส่งไปที่เซิร์ฟเวอร์ เมื่อเซิร์ฟเวอร์ทำการประมวลผลเสร็จเรียบร้อยแล้ว ก็จะส่งผลลัพธ์ในรูปแบบของโซบรีเคาส์ป้อนกลับมาให้ไคลเอนต์ หลังจากนั้นโซบรีเคาส์ไคลเอนต์ออบเจกต์ จึงจะทำการแปลและส่งแมสเสจที่เก็บผลลัพธ์ของออปเปอเรชัน นั้นกลับไปให้แอปพลิเคชันตามไคอะแกรมด้านล่าง

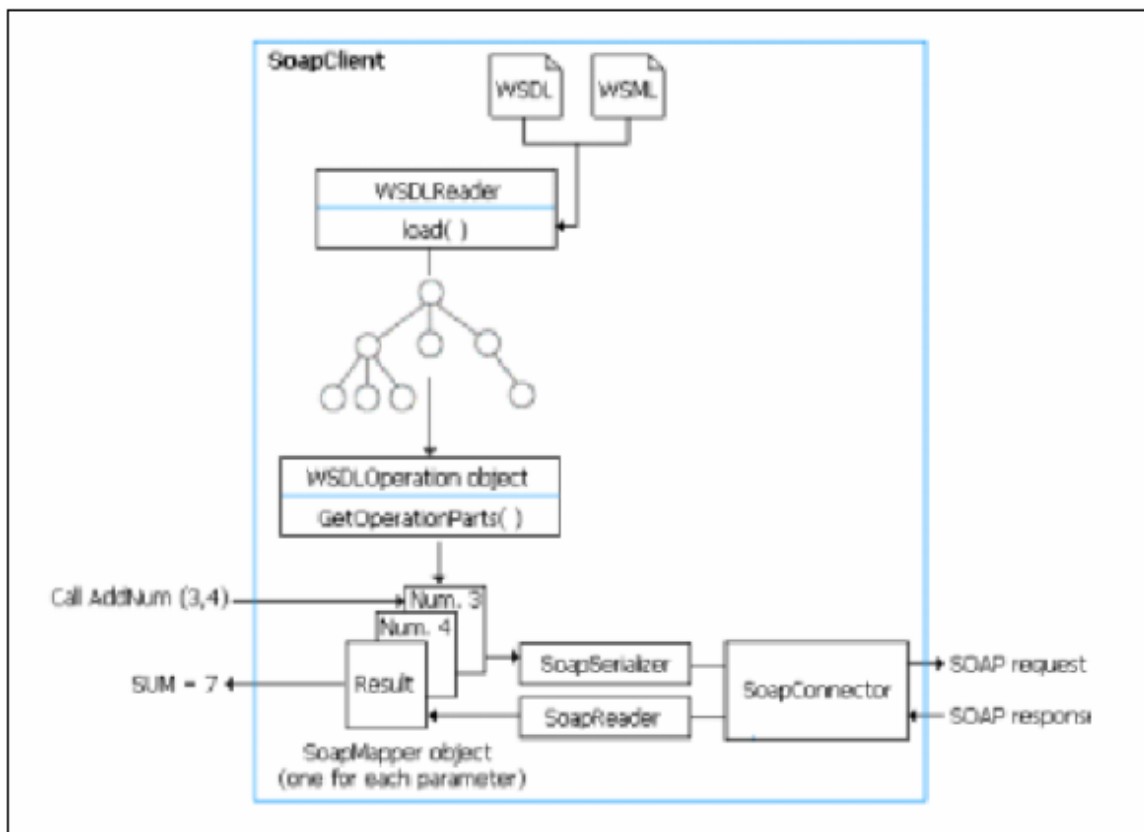


รูปภาพ 2-13 ตัวอย่างไคอะแกรมของการสร้างโซบรีเคาส์ไคลเอนต์ออบเจกต์

แอปพลิเคชันต้องอินสแตนทิเอตโซบรีเคาส์ออบเจกต์ (Instantiate Soap Object) ก่อนที่จะทำการเรียกใช้ออปเปอเรชันใดๆ (โซบรีเคาส์ไคลเอนต์ออบเจกต์ นี้ถูกสร้างเก็บไว้ในไฟล์ MSSOAP1.dll) โดยใช้เมธอด SoapClient.mssoapinit (ชื่อไฟล์ ดับเบิลยูเอสบีแอล, ชื่อเซอร์วิส, ข้อมูลพอร์ต) โดยข้อมูลพอร์ตจะใช้ในการแบ่งแยกช่องทางที่ใช้ในการส่งโซบรีเคาส์

การประมวลผลภายในโซปไคลเอนต์ออบเจกต์

มีการทำงานดังรูป



รูปภาพ 2-14 ตัวอย่างของการทำงานการประมวลผลภายในโซปไคลเอนต์ออบเจกต์

ดับเอสดีแอลรีดเดอร์ออบเจกต์ (WSDL Reader Object) จะทำการโหลดไฟล์ดับบีเอสดีแอล และไฟล์ดับบีเอสเอ็มแอลเข้าไปไว้ในคอม (DOM) แล้วทำการวิเคราะห์โดยดับบีเอสดีแอลรีดเดอร์ออบเจกต์ (WSDLReader Object) จะสร้างดับบีเอสแอลโอเปอเรชั่นออบเจกต์ (WSDLOperation object) เพื่อทำการเรียกใช้งานอ็อปเปอเรชั่น (ตามรูปนี้คือ AddNum(3, 4)) หลังจากนั้นดับบีเอสแอลโอเปอเรชั่น จะเรียกเมธอด GetOperationParts เพื่อดึงรูปแบบแมสเชจของอินพุตและเอาต์พุตของอ็อปเปอเรชั่นออกมาแล้วโซปไคลเอนต์ (SoapClient) จะสร้างโซปแมพเปอร์ออบเจกต์ (Soap Mapper Objects) ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้มาจากการเรียกใช้อ็อปเปอเรชั่น เข้าไว้ในออบเจกต์เหล่านี้ พอถึงขั้นตอนนี้ โซปซีริไรส์ออบเจกต์ (SoapSerializer Objects) จะสร้างโซปรีควีสแมสเชจ (SOAP Request Message) จาก โซปแมพเปอร์ออบเจกต์ (SoapMapper Objects) ที่เหมาะสมและส่งไปที่เซิร์ฟเวอร์

เมื่อเซิร์ฟเวอร์ประมวลผลเสร็จแล้วก็จะส่งโซปเรสพอนส์กลับไปที่ไคลเอนต์ โซปไคลเอนต์ออบเจกต์จะทำการโหลดผลลัพธ์ของอ็อปเปอเรชั่น ไปเก็บในโซปแมพเปอร์ออบเจกต์ที่เหมาะสมแล้ว

ส่งผลลัพธ์กลับไปให้แอปพลิเคชันของผู้ใช้

การสร้างโซบเซิร์ฟเวอร์ออบเจกต์ (SOAP Server Object)

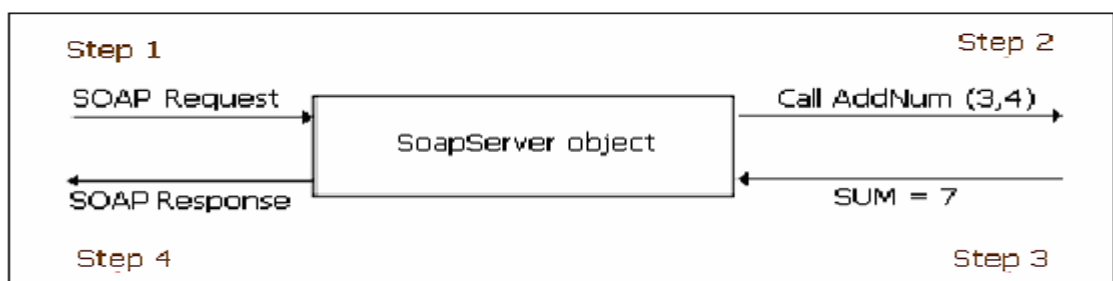
ส่วนทางฝั่งเซิร์ฟเวอร์เราสามารถกำหนดรูปแบบการรับโซบรีเควส (SOAP Listener) ได้ 2 รูปแบบ คือ อินเทอร์เน็ตเซิร์ฟเวอร์เอพีไอเซิร์ฟเวอร์ (Internet Server API Server) หรือ ไอเอสเอพีไอเซิร์ฟเวอร์ (ISAPI Server) และ แอกทีฟเซิร์ฟเวอร์เพจเซิร์ฟเวอร์ (Active Server Pages Server) หรือเอเอสพีเซิร์ฟเวอร์ (ASP Server)

```
<definitions>
...
<service name="DocSample1">
  <port name="DocSample1PortType" binding="tns:DocSample1Binding">
    <soap:address location=http://localhost/ DocSample1Test/ DocSample1.wsdl/>
  </port>
</service>
...
</definitions>
```

รูปภาพ 2-15 ตัวอย่างของโซบเซิร์ฟเวอร์ออบเจกต์

การทำงานทางฝั่งเซิร์ฟเวอร์

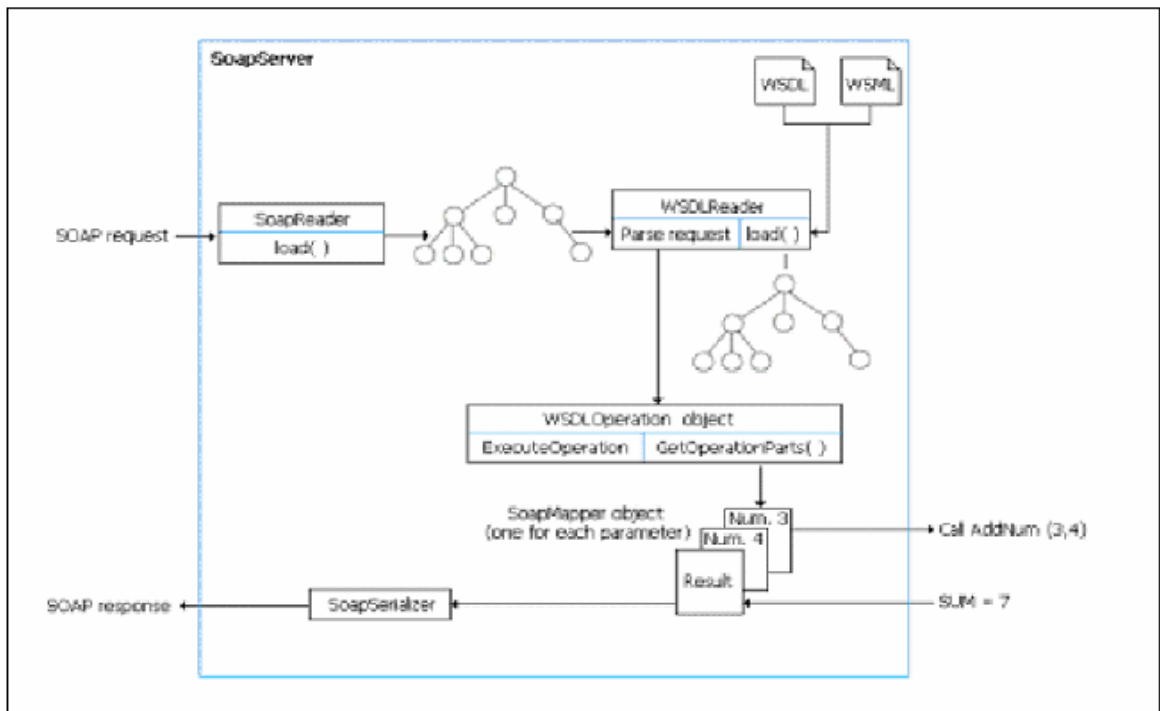
ไม่ว่าโซบรีเควสจะเรียกไปยังไอเอสเอพีไอเซิร์ฟเวอร์ หรือ เอเอสพีเซิร์ฟเวอร์ลิสเทนเนอร์ก็ตาม ข้อมูลที่รับส่งก็ยังคงเหมือนกัน เมื่อโซบเซิร์ฟเวอร์ออบเจกต์ ได้รับโซบรีเควสจากไคลเอนต์ ก็จะทำการแปลและเรียกไปยังเมธอด COM ที่ตรงกับออปเปอร์เรชั่นที่ต้องการ (ตามไดอะแกรมนี้คือเมธอด AddNum) หลังจากนั้นเมธอดที่ถูกเรียกใช้งานก็จะส่งผลลัพธ์กลับมาให้โซบเซิร์ฟเวอร์ออบเจกต์ซึ่งมันก็จะแปลงผลลัพธ์ให้อยู่ในรูปของ โซบเรสสปอนแล้วจึงส่งกลับไปที่ไคลเอนต์ ดังรูป (Server – Side Data Flow)



รูปภาพ 2-16 ตัวอย่างเซิร์ฟเวอร์ใช้ตัวดำดำโฟล

การประมวลผลภายในโซปเซิร์ฟเวอร์ออบเจกต์

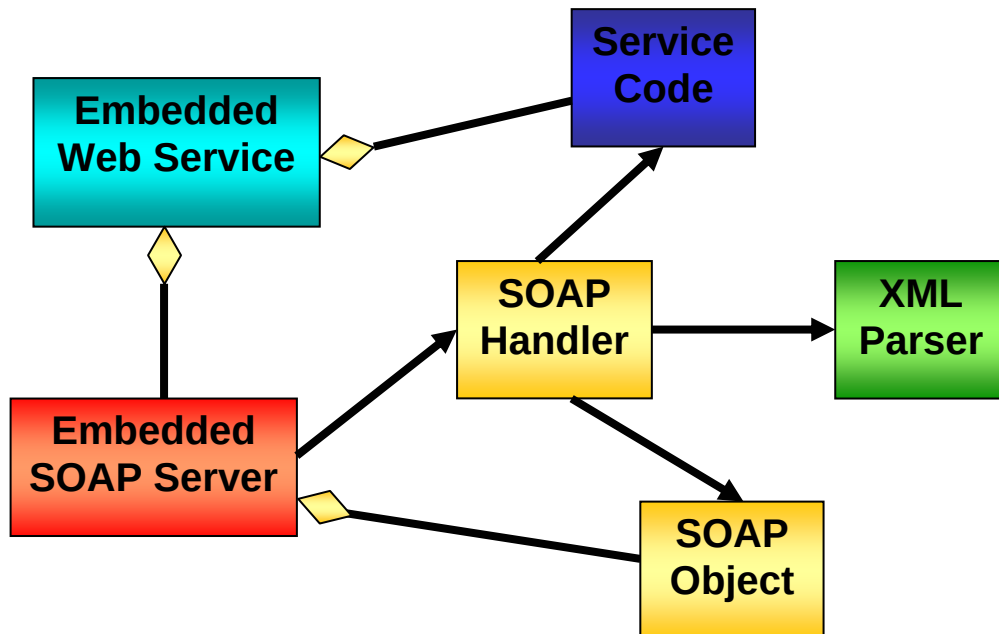
เมื่อได้รับโซปรีเควสจากไคลเอนต์แล้ว โซปรีดเดอร์ออบเจกต์ (SOAPReader Object) ก็จะโหลดรีเควสแมสเชงไปไว้ในดอมสตรักเจอร์ (DOM Struture) หนึ่ง ในขณะที่ดับบิวเอสดีแอลรีดเดอร์ (WSDLReader) จะโหลดไฟล์ดับบิวเอสดีแอล และไฟล์ดับบิวเอสเอ็มแอลไปไว้ในอีกดอมสตรักเจอร์หนึ่งตัว ดับบิวเอสดีแอลรีดเดอร์จะแปลรีเควสและสร้างดับบิวเอสดีแอลออปเปอร์เรชั่นออบเจกต์ขึ้นเพื่อใช้ในรีเควสออปเปอร์เรชั่น หลังจากนั้น ดับบิวเอสดีแอลออปเปอร์เรชั่นออบเจกต์ จะเรียกใช้เมธอด GetOperationParts เพื่อดึงรูปแบบแมสเชงของอินพุตและเอาท์พุทของออปเปอร์เรชั่นออกมา แล้วโซปเซอร์เวอร์จะสร้างโซปแมพเพอร์ออบเจกต์ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้จากการเรียกใช้ ออปเปอร์เรชั่น เข้าไว้ในออบเจกต์เหล่านี้ พอถึงขั้นตอนนี้ โซปเซิร์ฟเวอร์ออบเจกต์ จะทำการเรียกไปที่คอมเมธอด (COM method) ที่ตรงกับที่ร้องขอเข้ามา ดังรูป



รูปภาพ 2-17 ตัวอย่างของการทำงานการประมวลผลภายในโซปเซิร์ฟเวอร์ออบเจกต์

2.4 การออกแบบเว็บเซอร์วิสบนระบบฝังตัว จากโครงการปี 2545

2.4.1 โครงสร้างของเว็บเซอร์วิสบนระบบฝังตัว

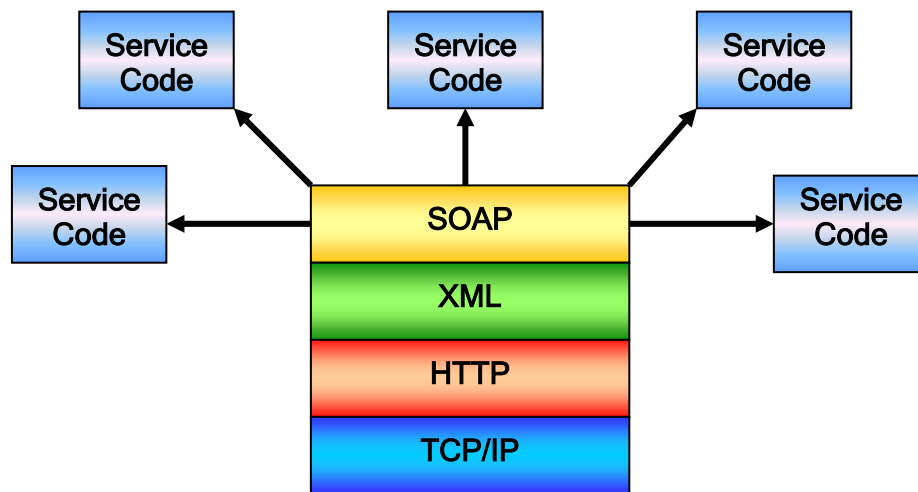


รูปภาพ 2-18 คอนเท็กซ์ไดอะแกรม (Context Diagram) ของเว็บเซอร์วิสบนระบบฝังตัว

สำหรับโครงสร้างภายในเว็บเซอร์วิสนี้มีส่วนที่สำคัญ คือ

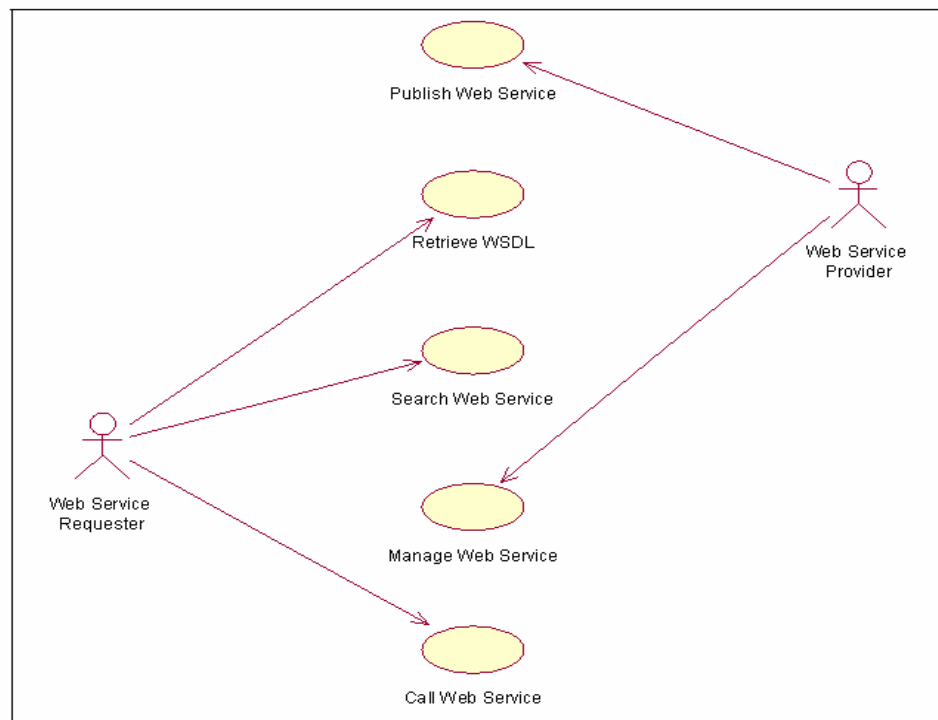
- โซบเซิร์ฟเวอร์บนระบบฝังตัว (Embedded Server) เป็นส่วนที่จัดการเซดเดอร์เอชทีทีพีและการติดต่อผ่านทางโพรโตคอลทีซีพีไอพี
- โซบแฮนเดิลเลอร์ (SOAP Handler) เป็นส่วนที่ทำหน้าที่จัดการโซบแมสเสจ ซึ่งจะแบ่งเป็น 2 ส่วนคือ ส่วนที่ทำหน้าที่จัดการโซบแมสเสจที่ได้รับมาเมื่อมีการเรียกใช้เซอร์วิสโดยถอดโซบแมสเสจ ให้เป็นรายละเอียดและข้อมูลอินพุตในการเรียกใช้เซอร์วิส และส่วนที่ทำหน้าที่จัดการสร้างโซบแมสเสจเพื่อตอบรับการเรียกใช้เซอร์วิสโดยได้นำเอาผลลัพธ์ที่ได้จากไฟล์เอาต์พุตที่ได้จากการเรียกเซอร์วิสมาเป็นโซบแมสเสจ
- เซอร์วิสโค้ด (Service Code) เป็นตัวแอปพลิเคชันที่อยู่บนตัวอุปกรณ์คอมพิวเตอร์ 86 ซึ่งจะถูกเรียกใช้งานโดยส่วนของโซบแฮนเดิลเลอร์
- โซบออบเจกต์ (SOAP Object) เป็นส่วนที่ทำหน้าที่เสมือนเป็นส่วนประกอบของโซบแมสเสจซึ่งจะถูกเรียกใช้โดยโซบแฮนเดิลเลอร์และเป็นส่วนหนึ่งของโซบเซิร์ฟเวอร์
- เอ็กซ์เอ็มแอลพาร์เซอร์ (XML Parser) จะทำหน้าที่ในการแปลเอกสารเอ็กซ์เอ็มแอล โดยที่จะถูกโซบแฮนเดิลเลอร์ นั้นทำการเรียกใช้โซบแฮนเดิลเลอร์ จะทำหน้าที่ในการจัดการโซบ โดยจะถูกเรียกใช้งานโดยโซบเซิร์ฟเวอร์ในระบบฝังตัว

2.4.2 โครงสร้างทางสถาปัตยกรรมของเว็บเซอร์วิสบนระบบฝังตัว



รูปภาพ 2-19 โครงสร้างทางสถาปัตยกรรมของเว็บเซอร์วิสบนระบบฝังตัว

2.4.3 ยูสเคสไดอะแกรมของระบบ (Use-Case Diagram)



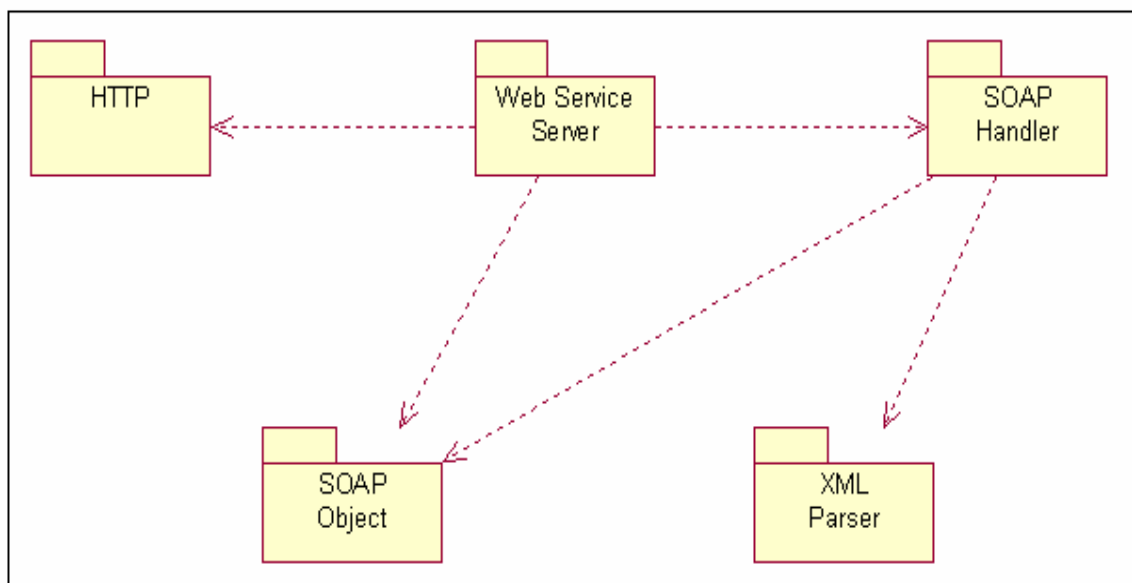
รูปภาพ 2-20 ยูสเคสไดอะแกรมของระบบ (Use-Case Diagram)

การร้องขอใช้บริการเว็บเซอร์วิส

Use Case	Call Web Service
ผู้ใช้	ผู้ให้บริการเว็บเซอร์วิส
เงื่อนไข	ผู้ให้บริการเว็บเซอร์วิสจะต้องร้องขอตามข้อกำหนดในเอกสารฉบับเอสดีแอลที่ได้มาจากตัวยูคิไอ และแอปพลิเคชันของผู้ใช้จะต้องสนับสนุนการทำงานของของโปรโตคอลโซบ และ เอชทีทีพี
ข้อมูลที่เกี่ยวข้อง	เมื่อได้รับเอชทีทีพีรีเควส มา จากนั้นแอปพลิเคชันโซฟต์แวร์เซอร์จะทำการแปลงให้อยู่ในรูปโซบแมสเชจแล้วค่อยทำการดึงรายละเอียดในการเรียกใช้บริการ เพื่อนำมาใช้ในการติดต่อกับตัวบริการนั้นๆ

ตาราง 2-4 รายละเอียดของการร้องขอใช้บริการเว็บเซอร์วิส

2.4.4 คอมโพเนนท์โคอะแกรมของระบบ (Component Diagram)



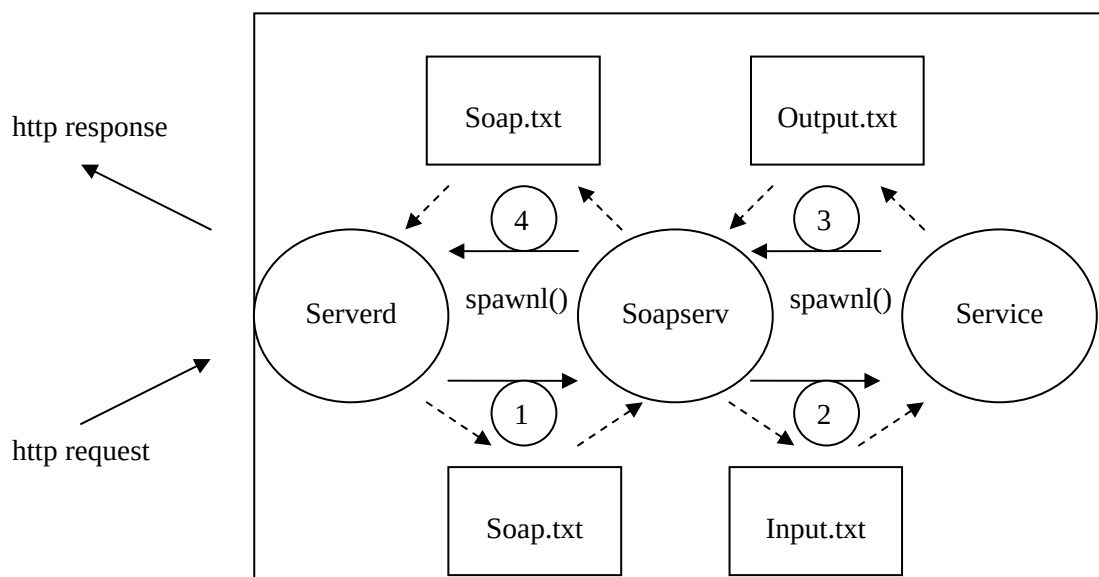
รูปภาพ 2-21 แสดงคอมโพเนนท์โคอะแกรม

2.4.5 กลไกการทำงานของระบบ

การทำงานของเว็บเซอร์วิสจะประกอบขึ้นจาก 3 ส่วน ได้แก่

1. **Serverd.exe** ทำหน้าที่เป็นเซิร์ฟเวอร์โปรแกรม ที่คอยรับรีเควส จากการขอใช้บริการผ่านพอร์ต 80 แล้วทำการสื่อสารข้อมูลตามโปรโตคอลเอชทีทีพี แล้วได้เอาท์พุตเป็นเอกสารไซบ จากนั้นก็รอผลลัพธ์จาก SoapServ.exe เพื่อส่งผลลัพธ์กลับไปยังผู้ที่ร้องขอต่อไปในขั้นตอนสุดท้าย
2. **SoapServ.exe** ทำหน้าที่รับเอกสารไซบ ที่ได้จาก Serverd.exe แล้วทำการดีซีเรียลไทร์ เอกสารไซบ ให้ได้เป็นข้อมูลของเซอร์วิสที่เรียกใช้ และพารามิเตอร์สำหรับการทำเซอร์วิสนั้นๆ จากนั้นก็รอผลลัพธ์จากเซอร์วิสแอปพลิเคชัน เพื่อนำส่งต่อไปให้ Serverd.exe ทำการส่งต่อไป
3. **Service application** เป็นแอปพลิเคชันที่มีฟังก์ชันการทำงานที่ผู้ร้องขอนั้น ต้องการเข้ามาขอใช้บริการ โดยแอปพลิเคชันนี้จะรับค่าพารามิเตอร์ในรูปแบบของการทำรีโมทโพรซีเจอร์คอล (Remote Procedure Call) มาทำการประมวล จากนั้นผลลัพธ์ที่ได้ก็จะถูกส่งไปยัง SoapServ.exe เพื่อทำการห่อหุ้มเอกสารกลับให้เป็นรูปแบบของเอกสารไซบ เพื่อส่งผลลัพธ์กลับไป

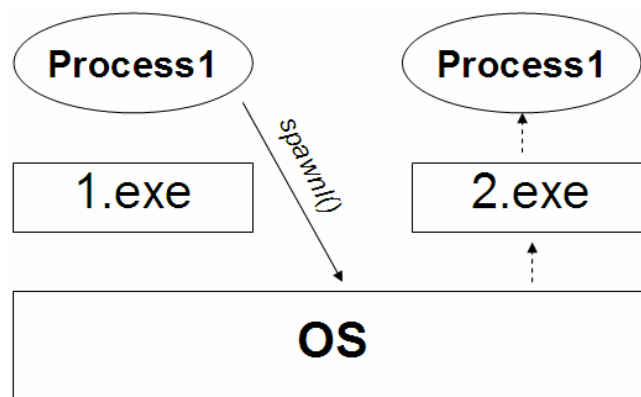
ขั้นตอนการทำงานของระบบเว็บเซอร์วิสแบบเดิม มีดังรูปต่อไปนี้



รูปภาพ 2-22 ขั้นตอนการทำงานของระบบเว็บเซอร์วิสแบบเดิม

การเรียกให้แอปพลิเคชันอื่นทำงานต่อ ด้วยฟังก์ชัน `spawnl()`

เป็นฟังก์ชันที่ใช้ในการเรียกแอปพลิเคชันอื่นขึ้นมาทำงานเป็น child process โดยเรียก path ของแอปพลิเคชันที่ต้องการ แล้วตามด้วย argument ไปด้วย ทำให้เหมือนการเรียกใช้ผ่าน shell หรือ dos โดยก่อนที่ระบบปฏิบัติการจะสร้างโปรเซสให้แอปพลิเคชันที่ถูกเรียกนั้นทำงาน ก็จะมีการบันทึกค่าตัวแปรแบบโลคอล (Local variable) และสถานะของโปรเซสที่ทำการเรียก เช่น รีจิสเตอร์ของซีพียู เก็บไว้ในสแตคของระบบ จากนั้นจึงค่อยสร้างโปรเซสของแอปพลิเคชันใหม่ขึ้นมาให้ทำงานเหมือนการเรียกแอปพลิเคชันแบบปกติตามคำสั่งที่โปรเซสที่ทำการเรียกส่งมาให้ และหลังจากที่โปรเซสของแอปพลิเคชันที่ถูกเรียกขึ้นมานั้นทำงานจนจบแล้ว ระบบปฏิบัติการก็จะนำค่าของตัวแปรและสถานะต่างๆของโปรเซสก่อนหน้า มาอยู่ในระบบเพื่อทำงานต่อจากจุดที่เรียกคำสั่ง `spawnl()` ต่อไป



รูปภาพ 2-23 การเรียกแอปพลิเคชันอื่นโดยใช้คำสั่ง `spawnl()`

2.4.6 ข้อเสียของลักษณะการทำงานแบบเดิม

- สามารถทำงานได้ทีละ 1 รีเควส ในการทำงานในช่วงเวลาหนึ่งๆ เท่านั้น
- รีเควสอื่นๆ จะเข้ามาทำงานได้หลังจาก request แรกได้รับการประมวลผลเรียบร้อยแล้วเท่านั้น
- หากโปรเซสใด มีการรอหรือทำการประมวลผลนานๆ ทั้งกระบวนการก็ต้องหยุดรอที่จุดนั้น ทำให้ไม่สามารถรับรีเควสอื่นเข้ามาทำงานได้
- หากโปรเซสใด เกิดความผิดพลาดในการทำงาน (corrupt) ขึ้นแล้ว โปรเซสอื่นทั้งหมดจะไม่สามารถทำงานต่อได้เลย
- ไม่สามารถลำดับความสำคัญ (priority) ของโปรเซสที่จะทำงานได้ ไม่ว่าเป็นงานที่สำคัญมากแค่ไหนก็ต้องรอให้งานที่ทำอยู่ก่อนหน้าทำเสร็จก่อน (non-preemptive)
- ไม่มีการส่งข้อมูลข้ามโปรเซสกัน โดยถ้าจะติดต่อกันระหว่างโปรเซสนั้น ต้องทำการบันทึกข้อมูลที่จะส่งให้กันลงในไฟล์ ทำให้เกิดโอเวอร์เฮดในการทำสูง

2.5 ระบบเรียลไทม์ (Real-Time System)

2.5.1 ความหมายของเรียลไทม์

ระบบเรียลไทม์ หมายถึง ระบบที่ทำงานเพื่อให้ได้ผลลัพธ์ โดยมีเงื่อนไข เวลา (time) และการคาดคะเนเวลาที่ใช้ในการทำงานได้ (predictable manner) โดยงานแต่ละงานจะต้องถูกทำให้เสร็จภายในระยะเวลาที่กำหนดไว้ และหากมีงานหลายๆงานเกิดขึ้นพร้อมๆกันแล้ว จะต้องมีการจัดแบ่งเวลาที่ใช้ในการประมวลผลของแต่ละงาน เพื่อให้เสร็จภายในระยะเวลาที่ต้องการ

2.5.2 ลักษณะของการทำงานแบบเรียลไทม์

- โต้ตอบกับระบบภายนอกได้ตลอดเวลา
- สามารถทำงานโดยรู้เงื่อนไขของเวลา
- ทำงานบนระบบที่มีทรัพยากรจำกัด

2.5.3 ประเภทของระบบเรียลไทม์

Hard Real-time : ระบบที่ทำงานโดยต้องการผลตอบสนองตามระยะเวลาที่กำหนด หากใช้เวลาเกินจากเส้นตายที่กำหนด การทำงานนั้นก็จะถือว่าทำงานผิดพลาด หรือล้มเหลวทันที

Soft Real-time : ระบบที่ไม่เกิดความเสียหายหรือล้มเหลวทันที หากไม่เสร็จทันตามระยะเวลาที่กำหนด แต่ระบบจะทำงานให้เสร็จหรือให้ได้ผลลัพธ์โดยเร็วที่สุดเท่าที่จะเป็นไปได้

2.5.4 ประโยชน์ของระบบเรียลไทม์

1. ระบบสามารถทำงานหลายๆงานได้พร้อมๆกัน
2. ระบบสามารถทำงานที่มีความสำคัญทางด้านเวลาสูงกว่า ให้ได้ผลลัพธ์ก่อนที่จะทำงานที่มีความสำคัญทางด้านเวลาต่ำกว่า
3. หากเกิดงานที่มีความสำคัญทางด้านเวลาสูงกว่า ระบบก็จะสามารถทำงานนั้นได้ทันที

2.6 ระบบปฏิบัติการแบบเรียลไทม์ (Real-Time Operating System)

เป็นระบบปฏิบัติการที่มีความสามารถในการทำงานแบบเรียลไทม์ ที่รองรับคุณสมบัติของการทำงานบนเงื่อนไขของเวลา โดยจะแตกต่างจากระบบปฏิบัติการทั่วไปโดยระบบปฏิบัติการแบบเรียลไทม์นี้จะมีการรับประกันระยะเวลานานที่สุดที่ระบบจะทำงานให้ได้ (worst case latency) ตัวอย่างความแตกต่าง เช่นหากเป็นระบบปฏิบัติการทั่วไปแล้ว หากเกิด interrupt จากภายนอกขึ้น ก็จะถูกนำไปใส่ต่อไว้ใน queue แล้วจะทำงานหลังจากทำการประมวลผล interrupt ตัวอื่นใน queue ก่อนหน้านั้นจนเสร็จสิ้นก่อน แต่หากเป็นระบบปฏิบัติการแบบเรียลไทม์ หาก interrupt ที่เกิดขึ้นนั้น มีความสำคัญกว่า งาน หรือ interrupt ที่ทำอยู่ในตอนนั้น ก็จะสามารถได้ทำงานก่อนทันที

2.6.1 เคอร์เนลของระบบปฏิบัติการแบบเรียลไทม์ (Real-Time Kernel)

เคอร์เนล (Kernel) เป็นส่วนกลางของระบบปฏิบัติการ ที่จะถูกนำขึ้นมาทำงานบนเมโมรี ก่อนเป็นอันดับแรก และจะอยู่ในเมโมรี ตลอดเวลา โดยจะทำหน้าที่ในการจัดการโปรเซส รวมถึงการจัดการใช้งานเมโมรี และการติดต่อดิสก์และ I/O ด้วย

สำหรับเคอร์เนลของระบบปฏิบัติการแบบเรียลไทม์นั้นจะต้องมีความสามารถดังต่อไปนี้

Service	Specification
Task Management	Preemptive Kernel Context Switching Mutual Exclusion Deadlock Embrace
Synchronization	Event Flags
Intertask Communication	Message Mailboxes Message Queues
Interrupt Management	Interrupt Latency Interrupt Response Interrupt Recovery
Time Management	Clock Tick Time Delay
Memory Management	Memory Control Block Memory Partitioning

ตาราง 2-5 ความสามารถของเคอร์เนลของระบบปฏิบัติการแบบเรียลไทม์

2.6.2 ซอฟต์แวร์ที่เลือกมาทำการศึกษาการทำงานแบบเรียลไทม์

Micro C / OSII : Real-time Kernel

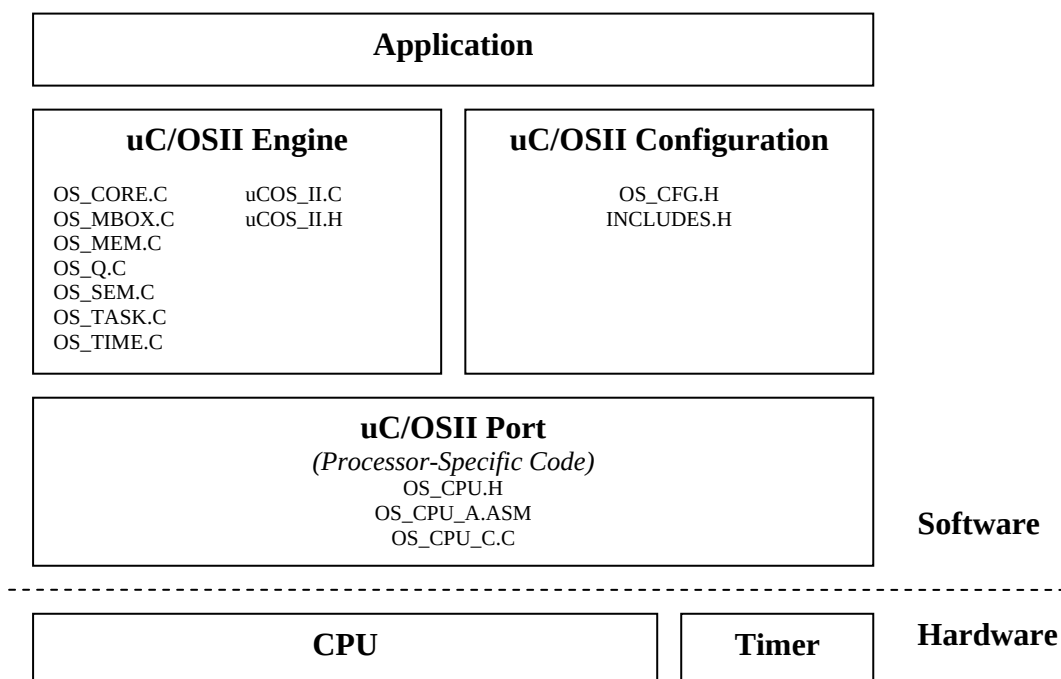
เป็นซอฟต์แวร์ประเภท opensource ซึ่งสามารถนำมาศึกษาและพัฒนาต่อได้โดยไม่ต้องเสียค่าใช้จ่าย มีลักษณะเป็นซอร์สโค้ดที่ประกอบไปด้วยส่วนของ engine ที่ทำหน้าที่เป็น real-time kernel และส่วนของฟังก์ชันที่สร้างขึ้นเพื่อให้นักพัฒนาสามารถใช้ในการจัดการกับงานต่างๆ ที่พัฒนาขึ้นได้เองอย่างครบถ้วน

Micro C / OSII มีความสามารถดังนี้

Source code	มีระบบการพัฒนาซอร์สโค้ดที่ดี มีการจัดทำเอกสารประกอบอย่างครบถ้วน และซอร์สโค้ดมีความง่ายต่อการศึกษา และพัฒนาต่อ
Portable	ซอร์สโค้ดได้ถูกพัฒนาขึ้นตามมาตรฐาน ANSI C ทำให้มีความง่ายในการนำไปพัฒนาและสร้างแอปพลิเคชันลงบนแพลตฟอร์มต่างๆ โดยทางผู้พัฒนา Micro C/OSII นั้นได้เตรียมเขียนส่วนของโปรแกรมภาษาแอสเซมบลี แยกเฉพาะตามไมโครโปรเซสเซอร์แต่ละแบบไว้แล้ว เพื่อนำไป port ได้ทันที
ROMable	สามารถนำพัฒนาเป็นส่วนหนึ่งของแอปพลิเคชันได้
Scalable	ใช้ทรัพยากรในระบบน้อย
Preemptive	เป็นแบบ fully preemptive real-time kernel ซึ่งทำให้สามารถประมวลผลงานที่มีความสำคัญสูง ให้เสร็จก่อนได้
Multitasking	สามารถรองรับงานได้ 64 task พร้อมๆกัน โดยมี 8 task เป็น task สำหรับระบบ ดังนั้นจึงสามารถมีงานที่นักพัฒนาสามารถสร้างขึ้นได้ 56 task
Deterministic	สามารถทราบถึงระยะเวลาในการประมวลผล task หนึ่งๆได้ แม้ว่าจะมีงานจำนวนมากทำงานอยู่ก็ตาม
Task Stacks	ใน stack ของแต่ละ task นั้นสามารถกำหนดเองได้ และสามารถตรวจสอบการใช้งาน stack ของแต่ละ task ได้อีกด้วย
Services	มี service ที่จำเป็นต่อการใช้งานอย่างครบถ้วน เช่น mailbox , queue , semaphore , fixed-sized memory partition เป็นต้น
Interrupt Management	สามารถเข้าแย่งการทำงานได้ หากมี priority สูงกว่า priority ของ task ที่ทำงานอยู่ในตอนนั้น แต่หากมี priority ต่ำกว่า และยังสามารถทำ nested interrupt ได้ 255 level
Robust and Reliable	มีความเชื่อถือได้ในเสถียรภาพ เนื่องจากถูกพัฒนามาเป็นระยะเวลานาน

ตาราง 2-6 ความสามารถของ Micro C/OSii

2.6.3 โครงสร้างของซอร์สโค้ดของ Micro C/OSii



รูปภาพ 2-24 โครงสร้างของซอร์สโค้ดของ Micro C/OSii

2.7 คอม86 (COM86)

ชุดพัฒนาคอม86 เป็นชุดเครื่องมือสำหรับใช้ในการพัฒนาอุปกรณ์ทางด้านระบบฝังตัวที่อยู่บนพื้นฐานการพัฒนาโดยใช้แพลตฟอร์มที่มีไมโครโปรเซสเซอร์ตระกูล x86 ซึ่งเป็นที่คุ้นเคยกับผู้ใช้งานในปัจจุบันเป็นส่วนประกอบหลักในการทำงาน

2.7.1 จุดเด่นของคอม86

2.7.1.1 ใช้ไมโครคอนโทรลเลอร์ที่มีความสามารถสูง

ภายในชุดพัฒนาคอม86 มีบอร์ดคอม86 ซึ่งได้เลือกใช้ไมโครคอนโทรลเลอร์ Am186CC ของบริษัทเอเอ็มดี (AMD) ซึ่งเป็นไมโครคอนโทรลเลอร์ขนาด 16 บิตทำงานอยู่ที่ความเร็ว 24 เมกกะเฮิร์ต (MHz) ทำให้มีความสามารถในการประมวลผลต่างๆเพิ่มมากขึ้นเมื่อเทียบกับการพัฒนาโดยใช้ไมโครคอนโทรลเลอร์ขนาด 8 บิตที่นิยมใช้กันอยู่ในปัจจุบัน นอกจากนี้ภายในตัวไมโครคอนโทรลเลอร์เองยังได้รวมเอาอุปกรณ์พื้นฐานต่างๆที่จำเป็นในการใช้งาน เช่น ดีแรมคอนโทรลเลอร์ (DRAM controller), อินเทอร์รัพคอนโทรลเลอร์ (Interrupt controller), ยูอาร์ที (UART) เข้าไว้ภายในตัวไมโครคอนโทรลเลอร์ซึ่งทำให้ง่ายต่อการออกแบบอุปกรณ์เพื่อทำเป็นสินค้าภายหลังการพัฒนาตัวอย่างเช่น การนำบอร์ดคอม86

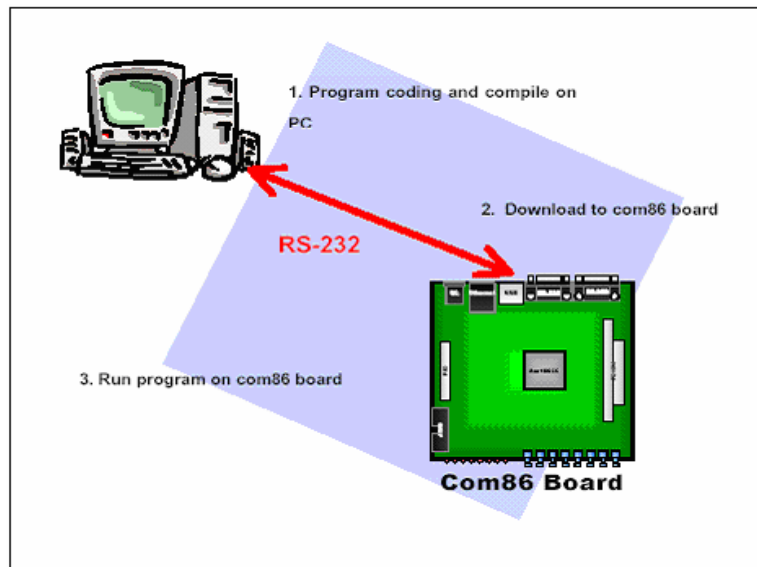
ไปพัฒนาเป็นอุปกรณ์อย่างหนึ่งที่ต้องการควบคุมการทำงานผ่านทางยูเอสบีพอร์ต (USB port) จะสามารถทำได้โดยง่ายเนื่องจากไม่ต้องการออกแบบวงจรควบคุมการทำงานในส่วนยูเอสบี นี้เพิ่มอีก เพียงแต่ออกแบบวงจรการทำงานส่วนอื่นที่ต้องการเพิ่มเติมทำให้ทุ่นเวลาในการพัฒนา เป็นต้น นอกจากนี้ภายในไมโครคอนโทรลเลอร์ยังเพิ่มส่วนโมดูล (Module) ทางด้านการสื่อสารต่างๆ เข้าไปภายในตัว ทำให้เป็นไมโครคอนโทรลเลอร์ที่มีความสามารถเด่นทางด้านการสื่อสารเหมาะกับการพัฒนาอุปกรณ์ที่ต้องใช้คุณสมบัติการสื่อสารต่างๆ ซึ่งจะสามารถทำได้ง่ายเมื่อเทียบกับการใช้ไมโครคอนโทรลเลอร์ขนาดเล็กแบบเดิมซึ่งมีข้อจำกัดต่างๆ อยู่มากมาย

2.7.1.2 มีเครื่องมือในการพัฒนาโปรแกรมมาก

เนื่องจากการพัฒนาโปรแกรมโดยใช้ชุดพัฒนาคอม86 นั้นเป็นเสมือนกับการพัฒนาโปรแกรมบนของคอมพิวเตอร์ส่วนบุคคลทั่วไป ดังนั้นจึงสามารถนำโปรแกรมเครื่องมือต่างๆ ที่ใช้พัฒนาโปรแกรมสำหรับคอมพิวเตอร์ส่วนบุคคลมาใช้งานได้มากมาย และโปรแกรมเครื่องมือเหล่านี้ก็เป็นพื้นฐานของนักพัฒนาจึงทำให้สามารถพัฒนาอุปกรณ์ได้อย่างรวดเร็ว ลดต้นทุนในการพัฒนาต่างๆ ได้เป็นจำนวนมาก นอกจากนี้ภายในชุดพัฒนายังมีชุดซอฟต์แวร์เครื่องมือต่างๆ ที่จำเป็นสำหรับช่วยในการพัฒนาโปรแกรมด้วยอีกทางหนึ่ง

2.7.1.3 ง่ายในการเรียนรู้และพัฒนา

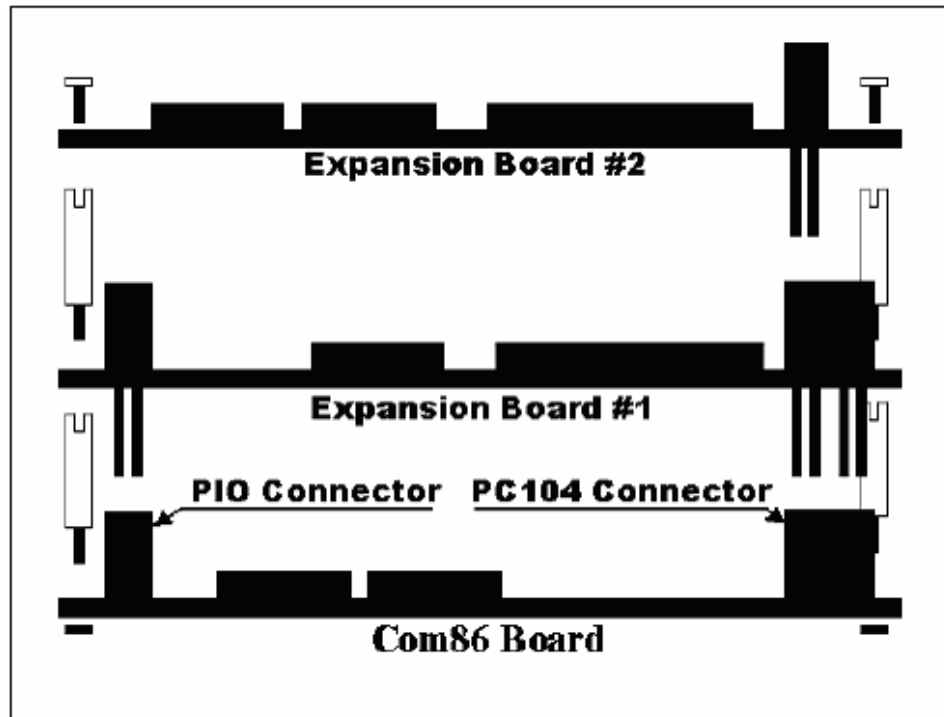
บอร์ดคอม86 ภายในชุดพัฒนาคอม86 นี้มีสถาปัตยกรรมของไมโครคอนโทรลเลอร์แบบ x86 ซึ่งเป็นสถาปัตยกรรมเดียวกับคอมพิวเตอร์ส่วนบุคคลทั่วไป โดยมีชุดคำสั่งที่เข้ากันได้กับชุดคำสั่งของ 80286 (Real mode) ทำให้ง่ายต่อการเรียนรู้ในการพัฒนาโปรแกรมต่างๆ และยังสามารถพัฒนาและทดสอบซอฟต์แวร์ต่างๆ ได้บนคอมพิวเตอร์ ก่อนที่จะทดสอบกับบอร์ดคอม86 ทำให้สามารถพัฒนาอุปกรณ์ทางด้วาระบบฝังตัวต่างๆ ได้อย่างรวดเร็ว ซึ่งรูปแบบการพัฒนาที่มีลักษณะเป็นไปดังภาพ



รูปภาพ 2-25 แสดงถึงรูปแบบการพัฒนาโปรแกรมโดยใช้บอร์ดคอม86

2.7.1.4 มีความสามารถในการเพิ่มขยายได้

ในส่วนของการเพิ่มขยายของฮาร์ดแวร์ (Hardware) สามารถทำได้โดยผ่านทางคอนเนคเตอร์ (Connector) ของบอร์ดในรูปแบบของมาตรฐาน (ซีพินหนึ่งศูนย์สี่) PC/104 ซึ่งจะมีการเชื่อมต่อแบบบัส อินเทอร์เฟซ (BUS Interface) แบบไอเอสเอบัส (ISA BUS) ขนาด 8 บิต คล้ายกับไอเอสเอบัส ที่มีอยู่ในคอมพิวเตอร์ส่วนบุคคลทั่วไป ซึ่งผู้พัฒนาสามารถออกแบบฮาร์ดแวร์ให้เป็นลักษณะของบอร์ดขยายและนำมาเชื่อมต่อกับบอร์ดคอม86 และเพียงเขียนซอฟต์แวร์ไดรเวอร์ (Software Driver) สำหรับควบคุมการทำงานของส่วนเพิ่มเข้ามาใหม่ซึ่งจะทำงานร่วมกับระบบปฏิบัติการในการรองรับการทำงานของแอฟพลิเคชั่นต่างๆที่เพิ่มเข้ามา นอกจากนี้ตัวบอร์ดคอม86 ยังสามารถเชื่อมต่อกับอุปกรณ์ภายนอกในลักษณะของพีไอโอ (PIO) หรือ โปรแกรมเมเบิลอินพุต/เอาต์พุต (Programmable Input/Output) เหมือนกันการใช้งานไมโครคอนโทรลเลอร์รุ่นอื่นๆได้ด้วยเช่นกัน



รูปภาพ 2-26 แสดงการเชื่อมต่อของบอร์ดขยายกับบอร์ดคอม86

2.7.2 แอปพลิเคชันเป้าหมายของชุดพัฒนา (Target Application)

จากความสามารถของไมโครคอนโทรลเลอร์ของบอร์ดคอม86 ทำให้การประยุกต์ใช้งานกับแอปพลิเคชันต่าง ๆ นั้น สามารถทำได้มากมาย โดยตัวอย่างแอปพลิเคชันต่างๆ ที่เหมาะแก่การพัฒนาโดยบอร์ดคอม86 มีดังนี้

2.7.2.1 เกี่ยวกับอุตสาหกรรม

ตัวอย่างของอุปกรณ์ควบคุมภายในโรงงาน เช่น การควบคุมเครื่องจักรจากระยะไกล การตรวจวัดสถานะต่างๆ ภายในโรงงานและส่งกลับไปยังศูนย์ควบคุมโดยการใช้บอร์ดคอม86 เป็นอุปกรณ์ที่ช่วยทำหน้าที่ในการสื่อสาร ควบคุมสถานะแวดล้อม หรือเครื่องจักรต่างๆ ภายในโรงงานและรายงานกลับไปยังศูนย์กลางตามกำหนดเวลา เป็นต้น

2.7.2.2 อุปกรณ์ยูเอสบี (USB Peripheral)

อุปกรณ์ยูเอสบีต่างๆ สามารถพัฒนาได้โดยการใช้บอร์ดคอม86 ซึ่งภายในบอร์ดคอม86 นี้มียูเอสบีคอนโทรลเลอร์ภายในตัวไมโครคอนโทรลเลอร์ Am186CC สามารถเชื่อมต่อเข้ากับคอมพิวเตอร์ได้ที่ความเร็ว 12 เมกกะบิตเปอร์เซ็ค (Mbps) ซึ่งการพัฒนาเป็นอุปกรณ์ ยูเอสบี ต่างๆ โดยบอร์ดคอม86 นี้สามารถทำได้โดยไม่ต้องมีการต่อวงจรทางด้านยูเอสบีใดๆ เพิ่มเติมอีก

2.7.2.3 เว็บเซิร์ฟเวอร์ (Embedded web server)

เอ็มเบดเดดเว็บเซิร์ฟเวอร์ (Embedded web server) สามารถสร้างได้บนบอร์ดคอม86 โดยไม่จำเป็นต้องเชื่อมต่อวงจรอื่นๆ เพิ่มเติม ซึ่งบอร์ดคอม86 นี้สามารถสื่อสารกับภายนอกผ่านทางอีเทอร์เน็ตพอร์ตที่อยู่บนบอร์ดที่ความเร็ว 10 เมกะบิตเปอร์เซ็ค ทำให้สามารถนำไปประยุกต์ใช้งานในกาออกแบบอุปกรณ์ควบคุมต่างๆ ผ่านเอ็มเบดเดดเว็บเซิร์ฟเวอร์ ที่ทำงานอยู่บนบอร์ดคอม86 ได้โดยง่าย

2.7.2.4 โรโบติก (Robotics)

บอร์ดคอม86 มีไมโครคอนโทรลเลอร์ที่มีประสิทธิภาพสูงในการทำงาน การพัฒนาซอฟต์แวร์สามารถพัฒนาได้ซับซ้อนมากยิ่งขึ้น การเชื่อมต่อขยายทางด้านฮาร์ดแวร์ต่างๆสามารถทำได้โดยง่าย เหมาะสำหรับการพัฒนาหุ่นยนต์ที่มีการทำงานที่ซับซ้อน

2.7.2.5 การควบคุมบ้านหรือสิ่งก่อสร้าง

อุปกรณ์สำหรับควบคุมระบบต่างๆภายในบ้าน หรืออาคารสามารถนำบอร์ดคอม86 ไปประยุกต์ใช้งานได้เช่นกัน โดยใช้เป็นส่วนในการควบคุมการทำงานของอุปกรณ์ต่างๆ หรือคอยตรวจสอบการทำงานของเซนเซอร์ (Sensor) ต่างๆ ภายในบ้าน และแจ้งไปยังเจ้าของบ้านเมื่อเกิดเหตุร้ายขึ้น

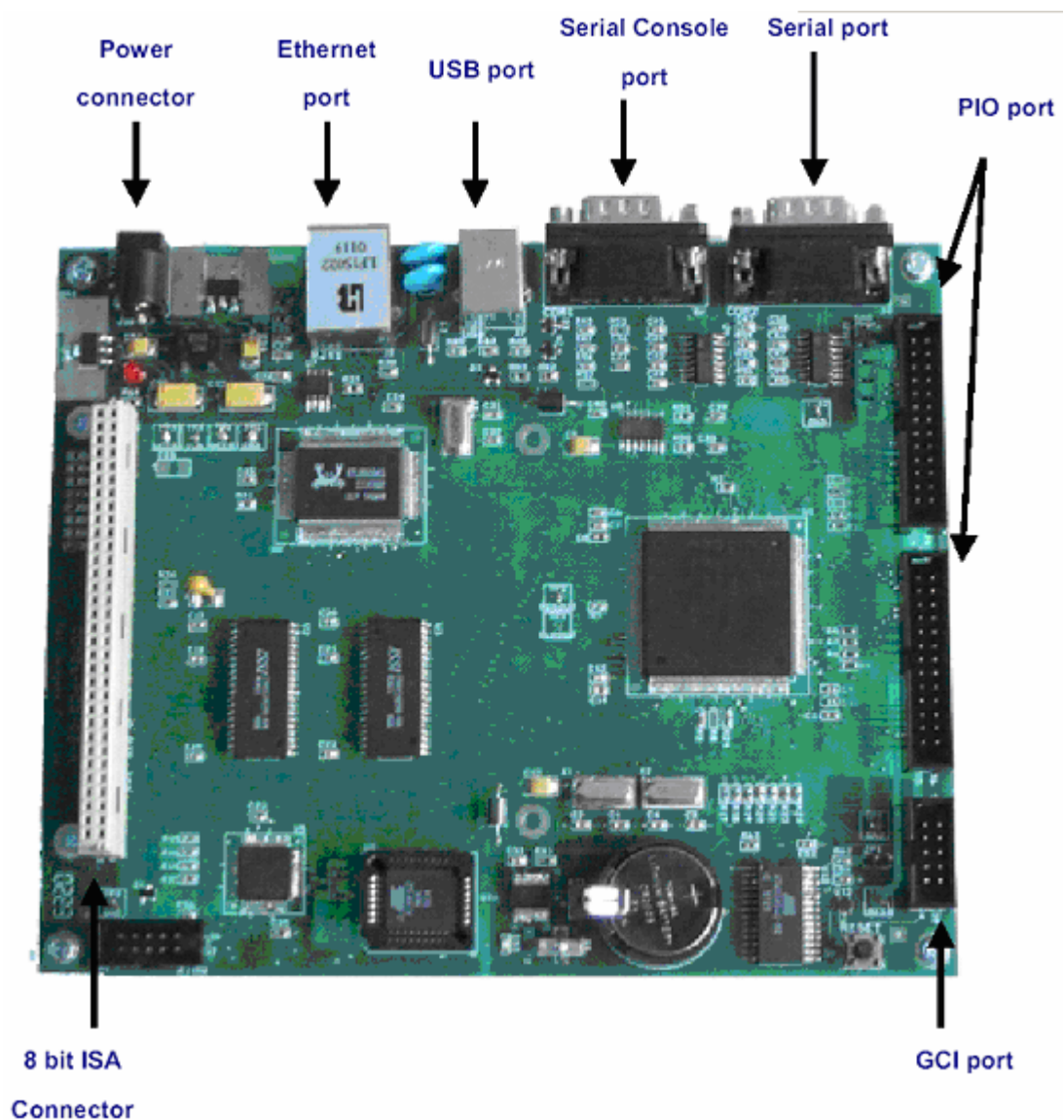
นอกจากนี้ตัวไมโครคอนโทรลเลอร์ยังรองรับการนำไปสร้างเป็นอุปกรณ์ต่างๆ ที่เกี่ยวข้องกับการงานทางด้านการสื่อสารต่างๆเช่น ไอเอสดีเอ็นโมเด็มแอนด์เทอร์มินัลอแดปเตอร์ (ISDN Modem and Terminal Adaptor), โลเอนด์เราเตอร์ (Low-end Router), การ์ดแอปพลิเคชัน (Line card application), เอ็กซ์ดีเอสแอลแอปพลิเคชัน (xDSL application), ดิจิตอลคอร์ดโฟน (Digital corded phone) เป็นต้น ซึ่งสามารถนำบอร์ดคอม86 ไปประยุกต์ใช้งานในการออกแบบพัฒนาได้เช่นกัน

2.7.3 คุณสมบัติของบอร์ดคอม86

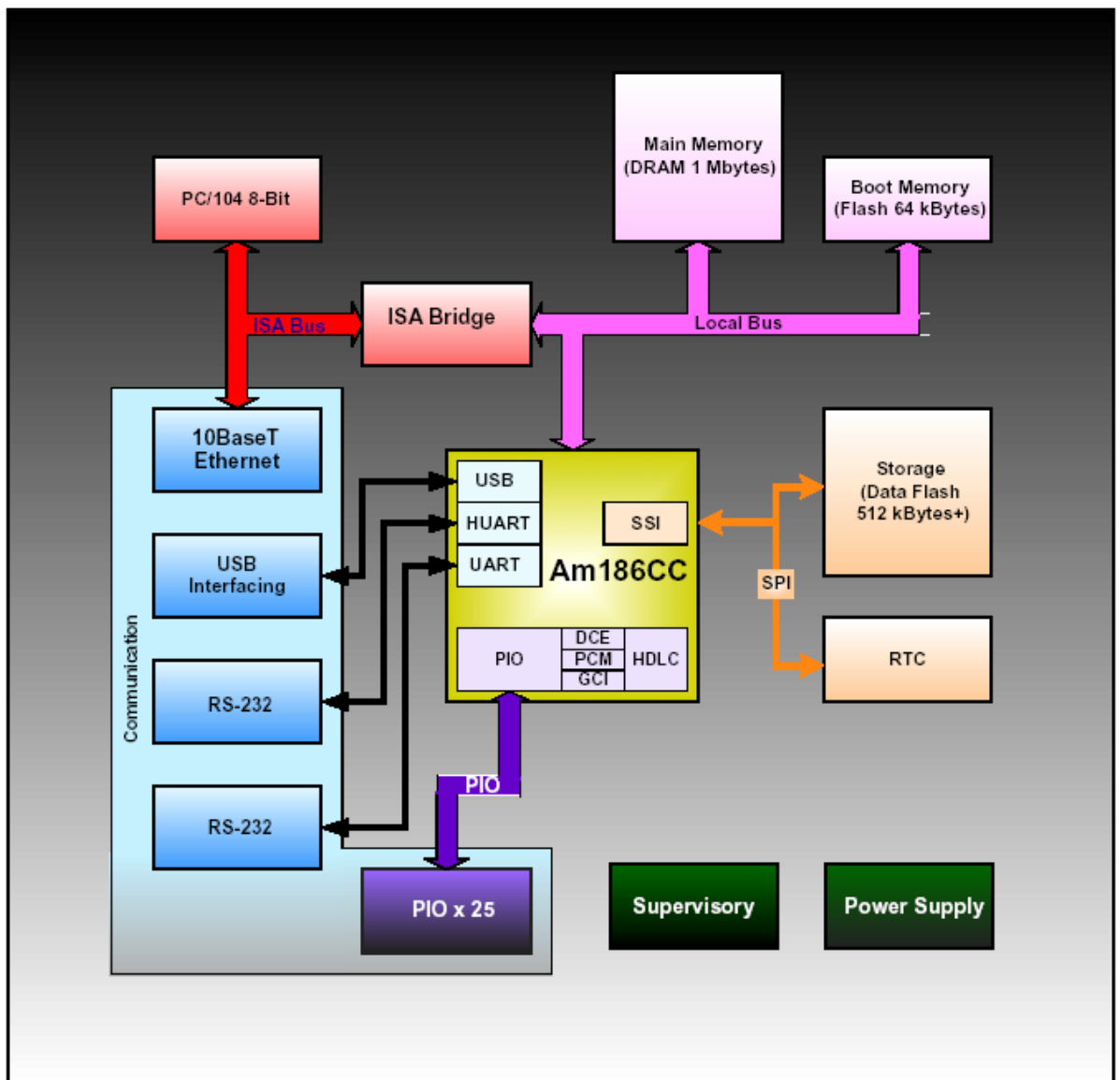
บอร์ดคอม86มีคุณสมบัติดังนี้

- ซีพียู เอซีหนึ่งแปดหกซีซี (CPU Am186CC)
- ดีแรม 1 เมกะไบต์ (1 MBytes DRAM)
- แฟลชไบออส 64 กิโลไบต์ (64 kBytes Flash BIOS)
- 512 kBytes Serial DataFlash® storage (upgradeable)
- Full duplex IEEE 802.3 compliant on board Ethernet controller with RJ45 connector
- USB peripheral controller version 1.1 with on-board type B USB connector
- Dual RS232-serial port with on-board DB-9 connector
- Four High level Data Link Control (HDLC) channels (multiplex with PIO)
- Four independent Time slot Assigner (TSAs)

- GCI Controller with on-board GCI port for external interface
- Three programmable 16-bit timers
- Real-Time clock
- Watchdog timer and reset controller
- Interrupt Controller (10 external sources)
- 25 programmable I/O
- 8 bit ISA bus in PC/104 form factor
- 3.3 V and 5 V on-board power supply
- LED Indicators for power supply and Ethernet



รูปภาพ 2-27 ตัวอย่างของภาพชุดพัฒนาคอม86



รูปภาพ 2-28 บล็อกไดอะแกรมแสดงส่วนประกอบต่างๆของคอม86